

Agentic Market Panic Simulator

*Bridging High-Frequency Market Microstructure and
Generative AI*

Benjamin Förster
Systems Engineering & Quantitative Finance

September 29, 2026

Abstract

The Agentic Market Panic Simulator (AMPS) addresses a critical gap in economic modeling: simulating the irrational, sentiment-driven dynamics of macroeconomic contagion at a microsecond scale. Utilizing a swarm of 400 generative Large Language Models (LLMs) interacting with a deterministic Limit Order Book (LOB), AMPS bridges behavioral finance and bare-metal high-performance computing. This book presents a cohesive architectural narrative detailing the empirical necessity of traditional finance datasets, the mitigation of computational bottlenecks via Singleflight request coalescing and zero-copy Apache Arrow data pipelines, and the hardware-level necessity of relying on SSE4.2 `popcnt` intrinsics over AVX-512 in heterogeneous CPU topologies. Through Matryoshka Representation Learning and 1.58-bit ternary quantization, AMPS achieves latency-optimized generative simulation on consumer-grade hardware.

Contents

Abstract	i
I Market Microstructure & Economic Theory	1
1 The Anatomy of Macroeconomic Contagion	2
1.1 The Limitations of Rational Agent Models	2
1.2 Information Asymmetry and Toxic Flow	4
1.2.1 The Mechanics of Adverse Selection	4
1.3 Liquidity Cascades and Margin Calls	4
1.4 The Imperative for High-Resolution Simulation	5
2 Mechanics of the Limit Order Book	6
2.1 The Order Queue Architecture	6
2.2 Order Types and Market Execution	6
2.2.1 Limit Orders (Liquidity Provision)	6
2.2.2 Market Orders (Liquidity Consumption)	7
2.2.3 Cancellation and Modification	7
2.3 Price-Time Priority and The Matching Engine	7
3 Algorithmic Market Making & Adverse Selection	9
3.1 The Function of the Market Maker	9
3.2 Inventory Risk and Mean Reversion	9
3.3 Adverse Selection and Toxic Flow	10
3.3.1 The Glosten-Milgrom Formulation	10
3.4 Limit Up-Limit Down (LULD) Circuit Breakers	11
4 Ecosystem Architectures: TradFi vs. Cryptocurrency	12
4.1 The Absence of Macroeconomic Stabilizers	12
4.1.1 Designated Market Makers (DMMs)	12
4.1.2 ETF Creation and Redemption Arbitrage	12
4.2 Leverage and the Liquidation Engine	13
4.3 The Data Quality Imperative	13
II High-Performance Systems Engineering	14
5 Bare-Metal Hardware Topography	15
5.1 The Heterogeneous Core Architecture	15

5.1.1	The Danger of Thread Migration	15
5.2	Strict NUMA-Style CPU Pinning	16
5.3	Memory Paging and the TLB Penalty	17
5.3.1	Transparent HugePages (THP)	17
6	Data-Oriented Design in Python	18
6.1	The Failure of Object-Oriented Python	18
6.2	Entity-Component-System (ECS) Architecture	18
6.3	Just-In-Time Compilation with Numba	19
6.3.1	Branchless SIMD Execution	19
7	Zero-Copy Inter-Process Communication	20
7.1	The Overhead of Serialization	20
7.2	Pure Python Shared Memory (/dev/shm)	20
7.3	The Apache Arrow C Data Interface	21
7.4	Lock-Free Ring Buffers and MESI Cache Protocols	21
8	High-Frequency Event Loops	22
8.1	The Multithreading Context-Switch Penalty	22
8.2	Cooperative Multitasking via <code>asyncio</code> and <code>uvloop</code>	22
8.3	Thundering Herd Mitigation (Singleflight Coalescing)	23
III	Cognitive Architectures & AI Topologies	24
9	Generative Agent Swarms	25
9.1	The 70B Macro Orchestrator (The God Agent)	25
9.2	The 400 Micro-Agents (The Swarm)	25
9.2.1	Narrative-to-Order Mapping	26
9.2.2	Bounded Rationality and Execution Friction	26
9.3	Cognitive Drift and Reflexion Loops	26
10	Semantic Caching & Discretization	28
10.1	The Concept of Semantic Caching	28
10.2	Matryoshka Representation Learning (MRL)	28
10.3	1-Bit Binary Quantization (BQ)	29
10.4	Ternary Weight Quantization (1.58-bit)	29
11	Hardware Acceleration for Semantic Search	31
11.1	Hamming Distance and Bitwise Operations	31
11.2	The AVX-512 vs. BMI2 Dilemma	31
11.3	Exploiting the SSE4.2 <code>popcnt</code> Intrinsic	32
12	Network Topologies of Panic	33
12.1	The Macro Information Bus (The Broadcast Graph)	33
12.2	The Social Contagion Bus (The Small-World Graph)	33
12.3	The Financial Execution Bus (The Bipartite Graph)	34
13	Non-Parametric Modeling & Causal Inference	35
13.1	The Limitations of Parametric Models	35
13.2	Kernel Ridge Regression (KRR)	35

13.3 Non-Parametric Instrumental Variables (NPIV)	36
IV Implementation & Epistemology	37
14 The BYOD Architecture and Data Ingestion	38
14.1 The NASDAQ LOBSTER Licensing Constraint	38
14.2 The "Bring Your Own Data" (BYOD) Solution	38
14.3 Alternative Datasets: ITCH-Parsed and hkgsas	39
14.4 Parsing to Memory: Bypassing Pandas	39
15 Stochastic Replay and Double Buffering	40
15.1 The Non-Determinism Problem	40
15.2 Double Buffering and Immutability	40
15.3 Zarr Telemetry Buffers	41
15.4 Engine Bypass for Deterministic Replay	41
16 Automated Epistemic Soundness Audits	42
16.1 Data-Flow Matrix Synchronization	42
16.1.1 Table-to-Trace Parity	42
16.2 Branchless Mask Verification	43
16.3 The Continuous Integrity Gate	43
V Operational Dashboards & Case Studies	44
17 Front-End UI Architecture	45
17.1 The Failure of Client-Side State	45
17.2 Binary WebSockets and HTML5 Canvas	45
17.3 Draft State and Scenario Loading	46
18 Zero-Latency WebSocket Streaming	47
18.1 The Threat of Network I/O Blocking	47
18.2 Decoupling via Redis PubSub	47
18.2.1 The Publisher (Simulation Engine)	47
18.2.2 The Subscriber (FastAPI Server)	48
19 The Data-Flow Matrix Case Studies	49
19.1 Case Study 1: The Liquidity Black Hole (Lehman Scenario)	49
19.1.1 Data-Flow Matrix Specifications: Lehman Shock	49
19.2 Case Study 2: The Flash Rally (Short Squeeze)	49
19.2.1 Data-Flow Matrix Specifications: Short Squeeze	50
19.3 Case Study 3: The Fat-Finger Anomaly	50
19.3.1 Data-Flow Matrix Specifications: Fat-Finger	51
VI Appendices	52
A Mathematical Derivations of LOB Metrics	53
A.1 Order Flow Imbalance (OFI)	53
A.2 Volume-Weighted Average Price (VWAP)	53

A.3	Bid-Ask Spread and Liquidity Voiding	54
B	Reference Implementation: Singleflight Coalescing	55
B.1	The Singleflight Mutex Map	55
B.2	Execution Metrics and GPU VRAM Preservation	56
C	Technical Hardware Optimization Blueprint	57
C.1	Workstation Topography & Core Memory Alignment	57
C.1.1	Baseline Hardware State & Memory Budget Allocation	57
C.1.2	P/E-Core NUMA Pinning & Thread Isolation Topology	58
C.1.3	Memory Paging Optimization: 2MB Transparent HugePages (THP) Strategy	60
C.2	Optimization Blueprint 1: Tiered KV-Cache VRAM Pinning Policy . . .	61
C.2.1	Agent Role Profiling & Asymmetric Frequency Analysis	61
C.2.2	Implementation Architecture: TieredVRAMManager	62
C.3	Optimization Blueprint 2: State Input Discretization & Singleflight Coalescing	63
C.3.1	Continuous Metric Discretization Pipeline	63
C.3.2	Singleflight Request Coalescing Engine Architecture	64
C.4	Optimization Blueprint 3: Asynchronous Non-Blocking Macro Ring Buffer	65
C.4.1	Decoupling External 70B God Agent API Calls	65
C.4.2	Linear Decay Fallback & Shock Degradation	67
C.5	Optimization Blueprint 4: SIRC Hamming Distance Threshold Relaxation	68
C.5.1	SIRC Architecture & SSE4.2 POPCNT Intrinsics	68
C.5.2	Threshold Window Relaxation & Secondary FP32 Verification . .	69
C.6	Implementation Verification, CI/CD Gates, & Execution Roadmap . . .	70
C.6.1	Automated Epistemic Soundness & Branchless AST Auditing . . .	70
C.6.2	Consolidated Hardware Optimization Matrix	71

Part I

Market Microstructure & Economic Theory

Chapter 1

The Anatomy of Macroeconomic Contagion

The study of market microstructure focuses on the intricate processes, temporal dynamics, and algorithmic outcomes of exchanging assets under explicitly defined, latency-sensitive trading rules [1]. To construct an Agentic Market Panic Simulator (AMPS), one must first thoroughly deconstruct the financial phenomena it seeks to replicate.

A market panic—or flash crash—is not merely a rapid depreciation of asset prices; it is a structural failure of market liquidity driven by information asymmetry, bounded rationality, and mechanical forced liquidations. This chapter provides a deep financial and theoretical foundation for the simulation, explaining why modeling irrationality requires generative Large Language Models (LLMs) rather than traditional rational-agent equations.

1.1 The Limitations of Rational Agent Models

In classical financial economics, specifically under the Efficient Market Hypothesis (EMH), actors are presumed to be perfectly rational utility-maximizers. Traditional macroeconomic models (such as Dynamic Stochastic General Equilibrium or DSGE models) rely heavily on representative agents who instantly update their beliefs using Bayes' theorem when presented with new information.

While these models are mathematically elegant and perform well during periods of equilibrium, they structurally fail during crises. Real-world market participants suffer from bounded rationality. They face cognitive limits in processing massive streams of high-frequency data, they succumb to emotional contagion, and their execution is constrained by technological latency.

Figure 1.1 illustrates the complex, non-linear feedback loops inherent in a flash crash. Classical mathematical models struggle to capture the *sentiment decay* phase—the precise moment when fear overrides financial logic. By replacing mathematical equations with stateful generative LLMs, AMPS introduces the necessary cognitive friction, allowing agents to experience confusion, panic, and irrational herding.

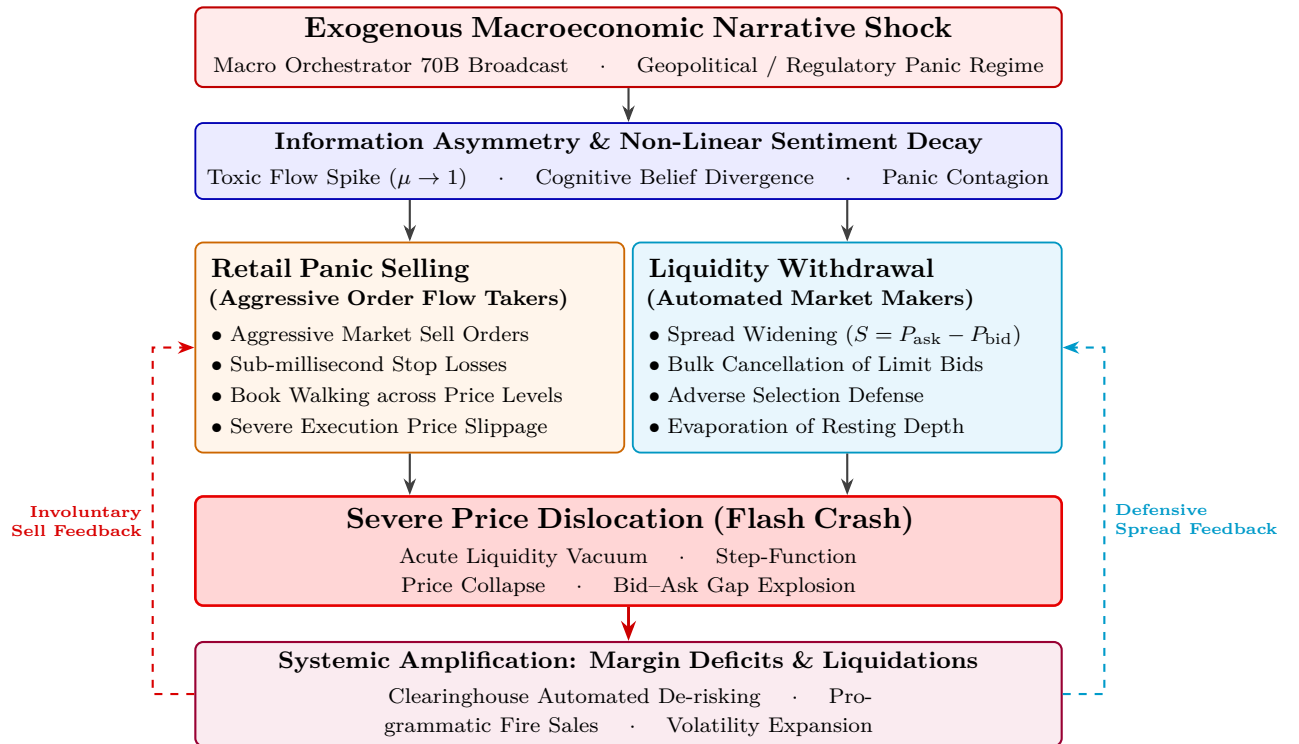


Figure 1.1: The Causal Loop of a Flash Crash. Exogenous shocks create information asymmetry. As retail agents panic sell, market makers defensively withdraw liquidity, causing a price dislocation that triggers mechanical margin calls, further exacerbating the sell-off.

1.2 Information Asymmetry and Toxic Flow

The core driver of market inefficiency is **Information Asymmetry**—a condition where some market participants possess material information before others.

In a high-frequency trading environment, information asymmetry manifests as **Toxic Flow**. Toxic flow occurs when an informed trader executes a large order against an uninformed algorithmic market maker (AMM). Because the informed trader knows the true underlying value of the asset is about to drop (due to an exogenous shock), they aggressively sell. The AMM, lacking this exogenous information, continues to buy the asset at the resting limit order prices. This mathematically guarantees a loss for the AMM, a concept formally known as **Adverse Selection**.

1.2.1 The Mechanics of Adverse Selection

Let V represent the true fundamental value of an asset, which is a random variable. The market maker quotes a Bid price B and an Ask price A . Let μ be the probability that an incoming trader is an *informed* trader (possessing toxic flow), and $(1 - \mu)$ be the probability the trader is *uninformed* (noise trader).

If the AMM blindly maintains a tight spread $S = (A - B)$ during a panic where $\mu \rightarrow 1$, the expected profit of the AMM becomes deeply negative:

$$E[\Pi_{AMM}] = (1 - \mu) \cdot \frac{S}{2} - \mu \cdot E[|V - B| \mid \text{Toxic}] \quad (1.1)$$

When the Macro Orchestrator God Agent in AMPS broadcasts a severe narrative shock, the internal μ (toxicity ratio) of the order flow spikes. The AMMs are programmed to detect this Order Flow Imbalance (OFI). To defend themselves against adverse selection, they must exponentially widen their spreads S or completely cancel their resting limit orders.

1.3 Liquidity Cascades and Margin Calls

The sudden withdrawal of resting liquidity by the AMMs removes the “floor” from the market. As the generative retail agents continue to submit aggressive Market Orders to sell their positions, these orders must traverse deeper into the Limit Order Book to find any remaining buyers, executing at progressively lower prices.

This rapid price degradation triggers a mechanical financial event: the **Margin Call**. Many institutional and retail traders utilize leverage (borrowed capital) to amplify their returns. Brokers enforce strict Maintenance Margin requirements. If the value of an agent’s portfolio drops below this threshold, the broker’s risk engine automatically and unconditionally liquidates the agent’s assets at the current market price to cover the debt.

This creates a catastrophic, self-reinforcing liquidity cascade:

1. Prices drop due to initial toxic flow.
2. Portfolio values cross the Maintenance Margin threshold.
3. Brokers inject massive, unconditional Market Sell Orders into an already depleted LOB.

4. Prices gap downwards instantly, triggering the next tranche of margin calls.

1.4 The Imperative for High-Resolution Simulation

To accurately model these micro-mechanics, it is fundamentally impossible to rely on End-of-Day (EOD) or minute-level Open-High-Low-Close-Volume (OHLCV) aggregates. OHLCV data systematically obfuscates the sub-second withdrawal of algorithmic liquidity and the exact tick-by-tick cascading of margin liquidations.

The AMPS project mandates the use of Level 2 (L2) market depth data or Level 3 (L3) message-by-message data. Only by reconstructing the exact density of the Limit Order Book can the generative agents interact with a mathematically sound financial reality.

In the subsequent chapters, we will deconstruct the exact mechanics of the Limit Order Book and explore the necessary transition into bare-metal systems engineering required to sustain 1,000 Ticks Per Second (TPS) of generative market activity.

Chapter 2

Mechanics of the Limit Order Book

The Limit Order Book (LOB) is the central nervous system of modern financial markets. Unlike traditional quote-driven markets where designated dealers dictate prices, order-driven markets rely on the continuous, decentralized aggregation of limit orders from diverse market participants. This chapter formalizes the mechanics of the LOB, providing the mathematical and structural blueprint necessary to build the simulation’s matching engine.

2.1 The Order Queue Architecture

At its core, a Limit Order Book is a dual-queue data structure. It aggregates the unexecuted intent of market participants into two distinct sides:

- **The Bid Side (B):** Composed of buyers. Orders are sorted in descending order of price. The highest price a buyer is willing to pay is the *Best Bid* ($P_{bid}^{(1)}$).
- **The Ask Side (A):** Composed of sellers. Orders are sorted in ascending order of price. The lowest price a seller is willing to accept is the *Best Ask* ($P_{ask}^{(1)}$).

The difference between the Best Ask and the Best Bid is the **Spread** ($S = P_{ask}^{(1)} - P_{bid}^{(1)}$). In a healthy, liquid market, the spread is extremely narrow (often a single tick). During a panic, the spread widens violently as liquidity evaporates.

2.2 Order Types and Market Execution

Market participants interact with the LOB through distinct order types, which fundamentally define whether they are *makers* of liquidity or *takers* of liquidity.

2.2.1 Limit Orders (Liquidity Provision)

A Limit Order specifies a price p and a volume v . If the order cannot be immediately matched against the opposing side, it rests in the LOB. These resting orders provide the “depth” of the market. Participants who submit limit orders are compensated with the

Level 2 (L2) Limit Order Book State Matrix ($\mathbf{L}_t \in \mathbb{R}^{2d \times 2}$) · FIFO Price-Time Priority			
Depth Level	Price (P)	Quantity (v)	Cumulative Depth / Liquidity (L)
Level 3 (Ask)	\$100.03	1,200 shares	$L_3 = 2,000$ shares
Level 2 (Ask)	\$100.02	500 shares	$L_2 = 800$ shares
Level 1 (Best)	\$100.01	300 shares	$L_1 = 300$
Inside Spread: $S = \$0.02$ (2 Ticks)		Mid-Price: $P_{\text{mid}} = \$100.00$	
← Buy Matches Ask (\$100.01)		Sell Matches Bid (\$99.99) →	
Level 1 (Best)	\$99.99	800 shares	$L_1 = 800$ shares
Level 2 (Bid)	\$99.98	250 shares	$L_2 = 1,050$ shares
Level 3 (Bid)	\$99.97	1,500 shares	$L_3 = 2,550$ shares

Figure 2.1: A stylized snapshot of a Level 2 (L2) Limit Order Book showing 3 levels of depth. Resting limit orders provide passive liquidity to the market.

ability to trade at their specified price, but they bear **Execution Risk**—the risk that the market moves away and their order never fills.

2.2.2 Market Orders (Liquidity Consumption)

A Market Order specifies a volume v but no price. It demands immediate execution and will aggressively match against the resting limit orders on the opposing side, starting at Level 1 and walking down the book (consuming depth) until the volume v is entirely fulfilled. Market orders guarantee execution but suffer from **Slippage**—the degradation of average execution price as deeper levels of the book are consumed.

2.2.3 Cancellation and Modification

High-Frequency Trading (HFT) environments are characterized by massive cancellation rates (often exceeding 95%). Algorithms continuously submit and cancel limit orders to probe for hidden liquidity or adjust to microsecond changes in momentum. The AMPS engine must support $O(1)$ cancellation latency to prevent simulation lockups.

2.3 Price-Time Priority and The Matching Engine

When an aggressive market order arrives, the exchange’s matching engine pairs it with resting limit orders. The universal standard for this process is **Price-Time Priority** (FIFO).

1. **Price Priority:** Orders with superior prices (higher bids, lower asks) are always executed first.
2. **Time Priority:** If multiple limit orders rest at the exact same price level, the order that was submitted chronologically earlier is executed first.

To simulate this mathematically, the LOB state at time t can be formalized as a sparse matrix or a discrete event vector. For generative agents, analyzing the raw event stream is

cognitively overwhelming. Instead, AMPS utilizes a quantized state matrix $\mathbf{L}_t \in \mathbb{R}^{2d \times 2}$, where d is the number of price levels tracked (e.g., $d = 10$ levels of depth), containing the aggregated volume and price at each level.

Understanding these mechanics is critical for Phase 2 of AMPS, where the matching engine is translated into contiguous memory arrays and executed branchlessly via Numba to achieve 1,000 TPS.

Chapter 3

Algorithmic Market Making & Adverse Selection

If AMPS relied entirely on generative Large Language Models (LLMs) to provide resting liquidity, the simulated market would instantly collapse. The inherent latency of LLM inference—even when heavily quantized—is fundamentally incompatible with the continuous, microsecond-level quoting obligations required to maintain a functional spread. Therefore, the simulation architecture mandates the deployment of deterministic **Algorithmic Market Makers (AMMs)**. This chapter explores the mathematical logic driving these AMMs and how their defensive behaviors act as the primary catalyst for flash crashes.

3.1 The Function of the Market Maker

A market maker is an algorithmic agent designed to provide continuous liquidity to the market. It does this by simultaneously quoting both a Bid and an Ask price. The market maker does not hold a directional opinion on the true value of the asset; instead, its goal is to earn the *Spread* (the difference between its Ask and its Bid) over thousands of round-trip trades.

If the market maker buys at \$99.99 and immediately sells at \$100.01, it captures \$0.02 in risk-free profit. However, this theoretical profit is constantly threatened by two primary risks: **Inventory Risk** and **Adverse Selection**.

3.2 Inventory Risk and Mean Reversion

An AMM must maintain a balanced inventory. If the market experiences a temporary influx of retail sellers, the AMM will continuously buy the asset, accumulating a massive long position. If the price permanently drops, the AMM suffers catastrophic capital losses.

To mitigate this, AMMs implement *Inventory Control*. As their inventory of an asset grows linearly, they skew their quotes exponentially to attract the opposite order flow. For example, if an AMM is dangerously long (holding too much of the asset), it will:

- Lower its Bid price (to discourage further buying).
- Lower its Ask price (to aggressively attract sellers and offload its inventory).

While inventory control is critical for normal market equilibrium, it breaks down entirely during a macroeconomic panic, leading us to the concept of Adverse Selection.

3.3 Adverse Selection and Toxic Flow

Adverse Selection occurs when a market maker trades against an informed counterparty. In the context of AMPS, when the Macro God Agent broadcasts a severe negative shock, the "Whale" agents receive this information instantly. They generate **Toxic Flow**—aggressive market sell orders driven by deterministic knowledge that the asset is overvalued.

3.3.1 The Glosten-Milgrom Formulation

The classic Glosten-Milgrom model provides the mathematical framework for understanding how AMMs react to toxic flow. Let $V \in \{V_{low}, V_{high}\}$ be the true fundamental value of the asset, which is known to informed traders but unknown to the AMM. The AMM only observes the order flow.

When the AMM observes an overwhelming ratio of sell orders to buy orders (an extreme Order Flow Imbalance, or OFI), its internal probability estimation that $V = V_{low}$ approaches 1.0.

To protect its capital, the AMM mathematically *must* widen its spread S . The formula for the required spread to break even against informed traders is directly proportional to the probability of trading with an informed trader (μ):

$$S \propto \frac{\mu}{1 - \mu} \cdot \sigma_V \quad (3.1)$$

where σ_V represents the volatility of the asset's true value. As $\mu \rightarrow 1$ (representing a pure panic where all incoming flow is toxic), the required spread $S \rightarrow \infty$.

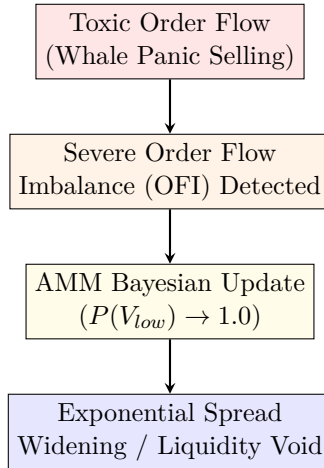


Figure 3.1: The deterministic logic pipeline of an Algorithmic Market Maker under toxic flow conditions.

3.4 Limit Up-Limit Down (LULD) Circuit Breakers

Because algorithms will perfectly execute Equation 3.1, a severe panic will cause the AMMs to instantly cancel all resting bids, creating a *Liquidity Void*. If a generative retail agent submits a market sell order into a liquidity void, the order will execute at a price of \$0.00, destroying the market simulation.

To prevent this, traditional finance (TradFi) exchanges implement regulatory **Circuit Breakers**, such as the Limit Up-Limit Down (LULD) mechanism. LULD explicitly pauses trading for 5 minutes if the price of an asset moves beyond a specified percentage band (e.g., 5% or 10%) within a rolling 5-minute window.

The inclusion of LULD dynamics is what strictly separates AMPS from unregulated cryptocurrency simulations, forcing the generative agents to comprehend regulatory halts and the physics of unexecutable panic.

Chapter 4

Ecosystem Architectures: TradFi vs. Cryptocurrency

When constructing the Agentic Market Panic Simulator, selecting the foundational dataset dictates the entire behavioral outcome of the generative swarm. The explicit exclusion of cryptocurrency data in favor of Traditional Finance (TradFi) equities is a critical architectural decision. This chapter details why the microstructure of cryptocurrency markets is fundamentally incompatible with modeling true macroeconomic contagion.

4.1 The Absence of Macroeconomic Stabilizers

In traditional equities, market panics are not purely unchecked free-falls; they are continuously mitigated (and occasionally uniquely exacerbated) by structural macroeconomic forces embedded in the exchange’s legal and financial framework.

4.1.1 Designated Market Makers (DMMs)

On traditional exchanges like the New York Stock Exchange (NYSE), specific firms are contractually obligated to act as Designated Market Makers. Unlike opportunistic High-Frequency Trading (HFT) firms that can instantly withdraw their liquidity during a panic (as discussed in Chapter 3), DMMs have regulatory obligations to maintain orderly markets. They are legally required to quote two-sided markets and absorb a certain amount of toxic flow to prevent instantaneous liquidity voids. Cryptocurrency exchanges lack this centralized, regulatory obligation entirely.

4.1.2 ETF Creation and Redemption Arbitrage

The most powerful stabilizing force in modern TradFi markets is the Exchange-Traded Fund (ETF) arbitrage mechanism. When an equity index (e.g., the S&P 500) experiences a sudden, sentiment-driven volatility spike, the ETF’s market price may disconnect from the Net Asset Value (NAV) of its underlying constituent stocks.

Authorized Participants (APs) immediately step in to exploit these fractional price discrepancies. If the constituent stocks are crashing faster than the ETF, APs will aggressively buy the underlying stocks (injecting massive stabilizing liquidity) and exchange them for ETF shares to sell at a premium. This continuous arbitrage firmly anchors individual

stock prices to the broader macroeconomic reality. Cryptocurrency markets, dominated by isolated, unbacked altcoins and perpetual futures, lack these deep, cross-asset arbitrage anchors.

4.2 Leverage and the Liquidation Engine

Cryptocurrency markets operate with unprecedented levels of unregulated leverage. Retail traders on platforms like Binance or Bybit can routinely access $100\times$ leverage through Perpetual Futures contracts.

When a minor price drop occurs in crypto, it instantly triggers the exchange’s automated liquidation engine, which market-sells the overleveraged positions. This causes a micro-flash crash that liquidates the next tier of leverage, resulting in a “scam wick”—a violent, instantaneous plunge and immediate recovery. These liquidations are entirely deterministic and mechanical, untethered from external macroeconomic reality or true information asymmetry.

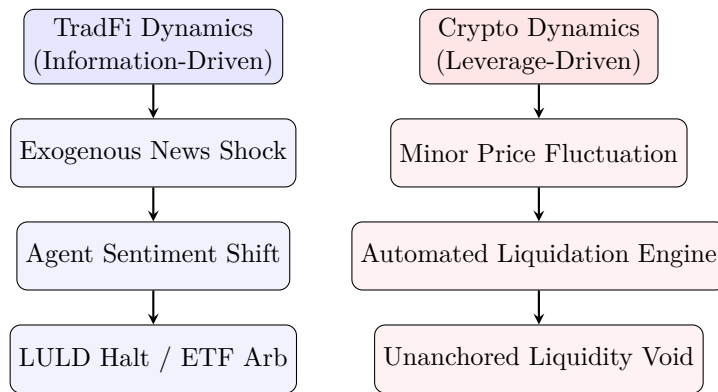


Figure 4.1: Divergent causal chains of market instability. TradFi panics are rooted in sentiment and bounded by regulation, whereas crypto panics are often purely mechanical liquidation cascades.

4.3 The Data Quality Imperative

Simulating a cryptocurrency flash crash yields behavioral patterns and order book depletion profiles that cannot be mathematically mapped to the structural realities of traditional macro indices. If the generative agents in AMPS were trained on crypto data, their internal Reflexion loops would over-optimize for surviving mechanical liquidation cascades rather than reasoning through complex, narrative-driven macroeconomic shocks.

Therefore, to construct a scientifically rigorous Agentic Market Panic Simulator, the foundational data must be derived exclusively from heavily regulated, order-driven fiat or equity markets. This constraint dictates our strict adherence to datasets like the Chinese LOB (hkgsas/LOB) and the Nasdaq LOBSTER dataset, which we will explore in detail in Part IV of this text.

Part II

High-Performance Systems Engineering

Chapter 5

Bare-Metal Hardware Topography

To simulate the microsecond-level mechanics of financial execution while simultaneously evaluating the cognitive states of 400 generative AI agents, the AMPS technology stack must abandon traditional cloud-native distributed microservices. Instead, it must dive into the physical realities of the underlying silicon. This chapter explains the bare-metal hardware topography required to guarantee strict determinism and ultra-low latency on a single consumer-grade workstation.

5.1 The Heterogeneous Core Architecture

The AMPS host machine is powered by an Intel Core i7-14700K processor. Historically, high-performance computing relied on Symmetric Multi-Processing (SMP) architectures, where every CPU core on the silicon die was physically identical, possessing the same instruction sets and cache hierarchies. The operating system’s Completely Fair Scheduler (CFS) could freely migrate threads across any available core without concern for architectural differences.

Modern Intel processors, beginning with the “Alder Lake” architecture and continuing into “Raptor Lake” (14700K), utilize a *heterogeneous hybrid topology*. The die is physically partitioned into two entirely distinct microarchitectures:

- **P-Cores (Raptor Cove):** 8 highly complex cores designed for maximum single-threaded throughput. They feature massive L2 caches (2MB per core), deep out-of-order execution windows, and advanced branch predictors. They consume significant wattage.
- **E-Cores (Gracemont):** 12 smaller, power-efficient cores designed for dense, highly threaded background tasks. They share a smaller L2 cache (4MB per 4-core cluster) and execute fewer instructions per clock (IPC), but provide massive aggregate throughput for parallel workloads.

5.1.1 The Danger of Thread Migration

In AMPS, the primary computational burden is fractured into two entirely different profiles. 1. The 400 micro-agents evaluating their generative semantic states. This is a memory-bound matrix multiplication workload running across the P-Cores. (Note: The 70B Macro Orchestrator God Agent is offloaded to an external API to preserve local

resources). 2. The Limit Order Book (LOB) matching engine. This is a latency-sensitive, highly-concurrent, branching workload.

Because Gracemont E-Cores and Raptor Cove P-Cores share the same L3 cache on the die, unconstrained execution of the memory-bound micro-agents across the entire processor would rapidly thrash the L3 cache, starving the LOB engine of its essential data. Furthermore, if the Linux scheduler is allowed to natively manage these threads, it will inevitably migrate threads across P-Core and E-Core boundaries. When a thread migrates from a P-Core to an E-Core, its local L2 cache context is invalidated. In a financial matching engine aiming for 1,000 Ticks Per Second (TPS), this cache thrashing introduces catastrophic *execution jitter*—stalling the LOB matching engine for milliseconds and destroying the temporal validity of the simulation.

To mitigate this shared L3 cache thrashing on consumer hardware, AMPS bypasses enterprise solutions like Intel RDT CAT. Instead, it utilizes strict Linux `cgroups` and `numactl` memory policies to pin the E-Core matching engine’s working set entirely within the localized 4MB L2 cache of the Gracemont cluster. By ensuring the LOB state arrays never exceed the L2 cache boundaries, the E-cores never need to fetch from the heavily contested L3 cache, granting them immunity from the P-core memory transversals.

5.2 Strict NUMA-Style CPU Pinning

To resolve the heterogeneous scheduling catastrophe, AMPS enforces rigid Non-Uniform Memory Access (NUMA)-style CPU pinning at the operating system level, explicitly bypassing the Linux CFS.

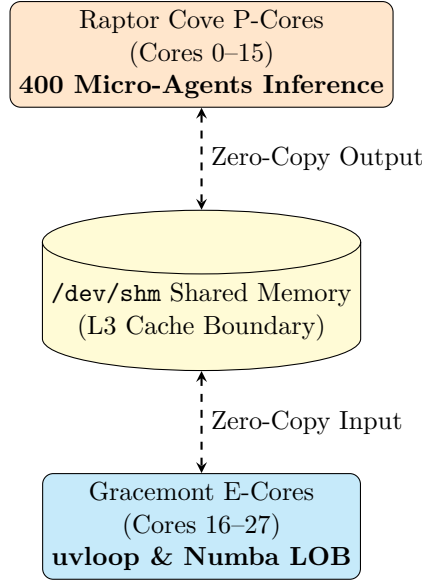


Figure 5.1: Strict CPU isolation boundaries. The micro-agents are locked to the P-Cores to execute matrix multiplication, while the high-frequency matching engine is locked to the E-Cores, preventing L2 cache thrashing and minimizing L3 contention.

By using the Linux `taskset` command (or `os.sched_setaffinity` in Python), the simulation binds the micro-agents strictly to Cores 0–15 (the 8 hyper-threaded P-cores). Simultaneously, the `uvloop` event loop orchestrating the data pipelines, alongside the

Numba JIT matching engine, is securely pinned to Cores 16–27 (the 12 E-cores). This physical isolation guarantees that the financial matching engine operates deterministically out of the Gracemont L2 cache, immune to the massive memory traversals occurring on the P-Cores.

5.3 Memory Paging and the TLB Penalty

Beyond core scheduling, memory mapping presents a severe bottleneck. Standard Linux memory allocation relies on 4 Kilobyte (KB) memory pages.

When the 400 micro-agents execute inference simultaneously, they allocate and continuously traverse over 12 Gigabytes of DDR5 RAM. Under a 4KB page paradigm, mapping 12GB of RAM requires the operating system to generate and maintain approximately 3 million individual virtual-to-physical page table entries.

The CPU relies on a hardware cache called the Translation Lookaside Buffer (TLB) to quickly resolve these virtual addresses to physical RAM addresses. However, the TLB on the i7-14700K can only hold a few thousand entries. Traversing 3 million pages forces the TLB to constantly evict and fetch new mappings from main memory, an expensive hardware interrupt known as a *TLB Miss*. The aggregate latency of millions of TLB misses is known as the TLB penalty, which can degrade swarm inference speeds by upwards of 30%.

5.3.1 Transparent HugePages (THP)

To eliminate the TLB penalty, the AMPS initialization sequence configures the Linux kernel to utilize **Transparent HugePages (THP)**. By utilizing the `madvise` system call within the LLM inference engine’s memory allocator (e.g., `llama.cpp`), the OS shifts the allocation block size from 4KB to 2 Megabytes (MB).

This structural shift reduces the page table footprint by a massive factor of 500 (from 3 million entries down to roughly 6,000). The TLB can now cache massive, contiguous blocks of the LLMs’ neural network weights without thrashing, ensuring that the micro-agents can synthesize their orders without lagging the master simulation clock.

Chapter 6

Data-Oriented Design in Python

To model a financial panic across 400 generative agents executing at 1,000 Ticks Per Second, the software architecture must fundamentally align with the hardware topography discussed in Chapter 5. Standard Python development practices are notoriously slow, entirely unsuited for sub-microsecond execution constraints. This chapter details the departure from Object-Oriented Programming (OOP) in favor of Data-Oriented Design (DOD), and the utilization of Just-In-Time (JIT) compilation to run native machine code directly from Python.

6.1 The Failure of Object-Oriented Python

In a standard Object-Oriented Programming paradigm, a software engineer might define the Limit Order Book using a class-based hierarchy. An `Order` class would contain attributes like `price`, `volume`, and `agent_id`. The `OrderBook` class would maintain a Python `list` or `dict` of these `Order` objects.

While conceptually intuitive for humans, this architecture is a disaster for the CPU. In Python, everything is an object. A simple integer is not merely 4 bytes of memory; it is a full C-struct (`PyObject`) containing a reference count, type pointers, and the value itself. When these objects are instantiated, Python's memory allocator scatters them randomly across the heap.

When the matching engine attempts to iterate through a list of `Order` objects to find a price match, the CPU encounters an array of pointers. It must dereference each pointer to find the actual object in memory. Because these objects are scattered, they cannot be loaded into the CPU's L1/L2 cache concurrently. Every pointer dereference causes a cache miss, forcing the CPU to stall while waiting for main memory. This completely destroys execution throughput.

6.2 Entity-Component-System (ECS) Architecture

To achieve maximum cache coherency and eliminate pointer chasing, AMPS adopts an Entity-Component-System (ECS) architecture, a pattern heavily utilized in high-performance video game engines but rarely seen in financial modeling.

ECS fundamentally decouples state (data) from behavior (logic):

- **Entities:** An entity (such as a specific order or a generative agent) is not an object. It is merely a unique integer ID (e.g., `Entity 402`).
- **Components:** Data is stored in dense, flat, contiguous arrays. For example, instead of an `Order` object holding a price, there is a massive global NumPy array called `OrderPrices`, where the index corresponds to the Entity ID.
- **Systems:** Logic resides in standalone functions that iterate over these contiguous arrays in tight, sequential loops.

By restructuring the Limit Order Book into an ECS format, all prices for all active orders exist sequentially in physical memory. When the CPU fetches the first price into the L1 cache, the hardware prefetcher automatically pulls in the next 15 prices sequentially, eliminating cache misses entirely.

6.3 Just-In-Time Compilation with Numba

While ECS provides the correct memory layout, standard Python loops still incur massive overhead due to dynamic typing and the Global Interpreter Lock (GIL). To execute the ECS arrays at bare-metal speeds, AMPS leverages **Numba**, an open-source JIT compiler that translates a subset of Python and NumPy code directly into fast machine code using the LLVM compiler library.

By applying the `@njit` (No-Python JIT) decorator to the matching engine systems, Numba completely bypasses the Python interpreter. The Python dynamic types are stripped away, and the logic is compiled into highly optimized, statically typed LLVM Intermediate Representation (IR), executing at speeds comparable to native C or C++.

6.3.1 Branchless SIMD Execution

The true power of coupling ECS with Numba lies in the ability to leverage Single Instruction, Multiple Data (SIMD) vectorization. When the matching engine needs to check if 1,000 active limit orders have crossed the spread, an Object-Oriented engine would use an `if/else` statement inside a loop.

In CPU architecture, `if/else` statements trigger the *branch predictor*. If the CPU mispredicts which path the `if` statement will take, it must flush its entire execution pipeline, a massive penalty.

Because our data is arranged in flat NumPy arrays, we can execute **Branchless Programming**. Instead of using an `if` statement, we perform vectorized boolean operations across the entire array simultaneously. Numba detects these array operations and compiles them into AVX2 or SSE4.2 hardware instructions, evaluating the price-time priority of thousands of orders simultaneously without ever triggering a branch misprediction. This guarantees deterministic, microsecond-level execution for the financial core of AMPS.

Chapter 7

Zero-Copy Inter-Process Communication

The AMPS engine is logically fractured: Python’s `asyncio` event loop orchestrates the 400 generative agents on one set of CPU cores, while the Numba JIT engine computes the Limit Order Book on another. To bridge these two distinct environments, the simulation requires high-throughput Inter-Process Communication (IPC). Traditional IPC mechanisms—such as WebSockets, gRPC, or Redis—introduce serialization and deserialization overhead that violates our microsecond latency budgets. This chapter details the implementation of true zero-copy IPC using shared memory and Apache Arrow.

7.1 The Overhead of Serialization

When two isolated processes communicate in standard Python, the data must be serialized into a byte stream (e.g., using `pickle` or JSON), transmitted over a local socket, and then deserialized back into Python objects by the receiving process.

For 400 agents requesting the complete LOB state matrix 1,000 times per second, this serialization/deserialization cycle consumes 99% of the CPU time, causing the simulation to grind to a halt.

7.2 Pure Python Shared Memory (`/dev/shm`)

To eliminate serialization, AMPS employs Python 3.8+ `multiprocessing.shared_memory`. This library maps directly to the POSIX `/dev/shm` hardware register on Linux, creating a block of physical RAM accessible to multiple isolated processes.

By defining a contiguous memory layout using a NumPy `dtype`, we achieve a highly structured payload. For example, an agent’s financial intent is represented as a 32-byte structure. This structure is mathematically aligned to fit exactly two intents into a single 64-byte L1 cache line on the Gracemont E-Cores, preventing partial cache line fetches.

Because Numba natively understands NumPy `dtype` configurations, it compiles array accesses directly into LLVM memory pointers. The Python agent writes its intent into the `shared_memory` block, and the Numba engine reads it directly via pointers. The data never leaves the shared memory, achieving true zero-copy data transit.

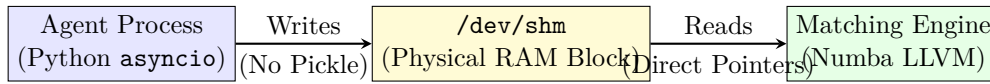


Figure 7.1: The Zero-Copy IPC pipeline. Data remains physically stationary in `/dev/shm`; only memory pointers are exchanged between the isolated processes.

7.3 The Apache Arrow C Data Interface

While `/dev/shm` works perfectly for simple intents, passing the massive, multi-dimensional LOB state matrix requires a more sophisticated memory format. AMPS leverages the **Apache Arrow C Data Interface**.

Arrow provides a standardized, columnar memory format that acts as a low-level lingua franca for sharing tabular data at runtime. Instead of relying on the heavy `PyArrow` library, AMPS utilizes Python’s `PyCapsule` objects.

The Numba matching engine constructs the LOB state in Arrow’s columnar format within `/dev/shm`. It then exposes the underlying C pointers of the `ArrowSchema` and `ArrowArray` structures. The Python agents consume these pointers via `PyCapsule`, instantly accessing the massive LOB matrix without a single byte of data being copied or serialized.

7.4 Lock-Free Ring Buffers and MESI Cache Protocols

Handling concurrent memory access normally requires Mutex locks to prevent data corruption. However, Mutex locks force the operating system to pause threads, introducing massive latency.

To resolve this, AMPS implements **Lock-Free Ring Buffers**. A ring buffer is a circular array of shared memory managed by atomic integer counters (a `head` pointer and a `tail` pointer). The matching engine pushes LOB updates to the `head`, and agents read from the `tail`.

When updating these atomic pointers, ownership of the CPU cache line holding the pointers is critical. If multiple CPU cores try to modify the pointer simultaneously, the CPU’s MESI (Modified, Exclusive, Shared, Invalid) cache protocol will force the cache line to continuously bounce back and forth between the L1 caches of the physical cores, destroying throughput (a phenomenon known as *False Sharing*).

By padding the atomic pointers so they sit on independent 64-byte cache lines, and utilizing strict LLVM memory fencing (`fence release` and `fence acquire`), AMPS guarantees lock-free, atomic memory visibility across the heterogeneous CPU cores with zero MESI cache bouncing.

Chapter 8

High-Frequency Event Loops

Managing 400 autonomous Large Language Model agents as they asynchronously evaluate market states, compute semantic trajectories, and post limit orders presents a severe orchestration challenge. This chapter details the departure from standard Python multithreading in favor of cooperative multitasking via `asyncio`, the integration of the `uvloop` C-extension, and the implementation of Singleflight request coalescing to prevent catastrophic GPU cache stampedes.

8.1 The Multithreading Context-Switch Penalty

A naive implementation of AMPS might spawn a unique operating system thread for every generative agent. With 400 agents, the Linux kernel must continuously pause and resume these 400 threads to give them all time on the physical CPU cores.

Every time the kernel switches from one thread to another, it incurs a *Context-Switch Penalty*. The CPU must save the exact state of its registers, clear the pipeline, load the new thread’s registers, and warm up the L1 cache. With 400 threads constantly demanding execution, the kernel wastes enormous amounts of CPU cycles merely juggling the threads, utterly destroying the simulation’s throughput and violating our microsecond latency budgets.

8.2 Cooperative Multitasking via `asyncio` and `uvloop`

To resolve this, AMPS abandons OS-level multithreading entirely in favor of **Cooperative Multitasking**. Python’s `asyncio` module operates on a single OS thread. Instead of the kernel forcibly interrupting agents, agents explicitly yield control of the CPU when they are waiting for I/O (such as waiting for the GPU to finish LLM inference or waiting for a timer to expire).

While the default Python `asyncio` event loop is a massive improvement over multithreading, it is written in pure Python and remains relatively slow. AMPS replaces the default loop with `uvloop`.

`uvloop` is a Cython wrapper around `libuv` (the ultra-fast C library that powers Node.js). Under the hood, `uvloop` hooks directly into the operating system’s lowest-level I/O notification facilities—specifically the `epoll` system call on Linux. This paradigm shift

allows the single Python thread to monitor thousands of agent states simultaneously in the C-kernel with practically zero polling overhead.

8.3 Thundering Herd Mitigation (Singleflight Coalescing)

Even with `uvloop`, AMPS faces a critical systems engineering crisis during a simulated market panic.

When the 70B Macro God Agent broadcasts a severe, systemic market shock (e.g., “Major Bank Files for Bankruptcy”), all 400 micro-agents wake up simultaneously. They all instantly attempt to query the local RTX 5070 Ti GPU for generative inference to decide whether to panic sell. This instantaneous, massive spike in I/O demand is known as a **Cache Stampede** or a “Thundering Herd.”

Sending 400 identical LLM prompts to the GPU simultaneously will exhaust the 16GB of VRAM and crash the CUDA kernels. To mitigate this fatal bottleneck, AMPS implements the **Singleflight** request coalescing pattern.

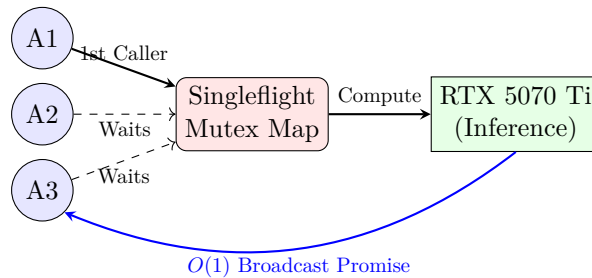


Figure 8.1: Mechanics of Singleflight coalescing. The first agent locks the operation and executes it. All subsequent agents yield their context and await the broadcast result, completely flattening GPU utilization spikes.

Mechanistically, Singleflight operates as follows within the `asyncio` loop:

1. **First Caller:** Agent 1 attempts to run inference based on the market shock. It acquires a non-blocking mutex lock, sees no active execution for this prompt, and creates a new `asyncio.Future` (a Promise). It registers this Promise in a global dictionary and yields control to wait for the GPU.
2. **Duplicate Callers:** Microseconds later, Agents 2 through 400 wake up and acquire the mutex. They detect the existing Promise in the dictionary, increment a duplicates counter, and instantly yield their execution context back to `uvloop`.
3. **Result Sharing:** When the GPU finishes processing Agent 1’s prompt, Agent 1 resolves the Promise. This acts as a broadcast signal that instantly wakes the 399 sleeping agents in $O(1)$ time and hands them the zero-copy result.

This mechanism completely flattens CPU and GPU utilization during massive volatility spikes, preventing the simulation from crashing exactly when the panic dynamics are most critical.

Part III

Cognitive Architectures & AI Topologies

Chapter 9

Generative Agent Swarms

While the matching engine computes the mathematical reality of the simulation, the behavior of the market is driven entirely by the cognitive decisions of generative Large Language Models (LLMs). This chapter details the hierarchical architecture of the Agent Swarm, dividing the computational load between a massive, omniscient Macro Orchestrator and hundreds of localized, boundedly-rational micro-agents.

9.1 The 70B Macro Orchestrator (The God Agent)

At the apex of the simulation hierarchy sits the Macro Orchestrator, often referred to as the "God Agent." Powered by a massive 70-Billion parameter language model (e.g., Llama 3 70B), this agent is entirely decoupled from the high-frequency trading loop. It operates on a slower, macro timescale (e.g., ticking every 10 seconds).

The primary responsibility of the God Agent is the synthesis of **Exogenous Narrative Shocks**. By continuously ingesting macroeconomic data, simulated geopolitical events, and historical templates from datasets like FinQA, the God Agent constructs coherent, cascading crises.

For example, rather than simply instructing agents to "sell," the God Agent generates a detailed SEC 8-K filing indicating systemic fraud at a major financial institution. This narrative is then broadcast to the ecosystem, forcing the micro-agents to independently parse the text, evaluate their portfolio exposure, and derive the logical financial action (panic selling).

Because of its massive size, the 70B model requires over 40GB of VRAM/RAM for inference, making it impossible to run locally on a 16GB workstation GPU alongside the micro-agents without memory starvation. Therefore, the God Agent's inference is strictly offloaded to an external ultra-low latency API (e.g., Groq). It passes its generated output back down to the local simulation via high-speed WebSockets.

9.2 The 400 Micro-Agents (The Swarm)

Beneath the God Agent exists a swarm of 400 autonomous micro-agents. These agents act as the retail traders, institutional whales, and momentum algorithms that generate order flow.

Due to the extreme constraint of running 400 models on a single RTX 5070 Ti (16GB VRAM), these micro-agents are instantiated using ultra-lightweight models (e.g., Qwen1.5-0.5B parameters). By applying 1.58-bit ternary quantization (detailed in Chapter 10), each agent's active memory footprint is compressed to mere megabytes, allowing all 400 to fit safely into local VRAM.

9.2.1 Narrative-to-Order Mapping

While the God Agent generates unstructured natural language (e.g., a simulated SEC 8-K), this text cannot be directly fed into the Limit Order Book. The micro-agents serve as the bridge. They ingest the narrative shock and are forced to output a strictly typed JSON schema (enforced via constrained decoding grammar).

The output schema strictly mandates:

1. **action**: [BUY, SELL, HOLD]
2. **price_offset**: Integer ticks from the current Best Bid/Ask.
3. **volume**: Integer number of shares based on their portfolio size.

This JSON is parsed by the Python backend and instantly translated into a discrete C++ data struct, which is then injected into the Numba JIT order matching engine.

9.2.2 Bounded Rationality and Execution Friction

In reality, retail traders do not possess perfect information, nor do they process information instantly. The micro-agents are programmed with **Bounded Rationality**. They receive delayed, noisy subsets of the Macro Shock. Furthermore, their prompts are injected with distinct psychological profiles (e.g., Risk-Averse, Momentum-Chasing, Contrarian).

When a shock occurs, a Momentum-Chasing agent might aggressively market-sell its entire portfolio instantly. A Contrarian agent might interpret the massive sell-off as a temporary liquidity gap and attempt to buy the dip, placing limit orders deep in the book.

9.3 Cognitive Drift and Reflexion Loops

A critical failure mode in multi-agent LLM simulations is **Model Collapse** or *Behavioral Drift*. Over thousands of continuous interactions, agents often spiral into nonsensical loops of repetitive trading, buying and selling the exact same asset back and forth instantly, completely draining liquidity without any rational justification.

To combat this, AMPS implements **Reflexion Loops**. Reflexion is a cognitive prompting architecture where the agent critiques its own past actions before committing to a new one.

Before a micro-agent submits a trade to the zero-copy `/dev/shm` bus, it must evaluate its *Proposed Intent* against a sliding window of its past actions and its current Profit and Loss (PnL).

If the internal critique deems the action overly repetitive (e.g., "I have sold this asset 5 times in the last 10 seconds without new information"), the agent self-corrects and suppresses the trade. This injects necessary behavioural friction into the system, ensuring that the simulated trading volume accurately mirrors human hesitation and risk management.

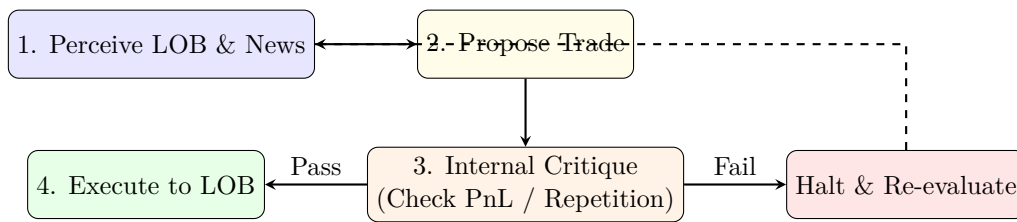


Figure 9.1: The Reflexion Loop. Agents mathematically evaluate the rationality of their own outputs against their historical PnL before executing, preventing recursive model collapse.

Chapter 10

Semantic Caching & Discretization

Even with the implementation of Singleflight request coalescing (Chapter 8), executing full LLM inference for 400 agents at 1,000 Ticks Per Second remains physically impossible on consumer hardware. To achieve true high-frequency scale, the simulator must learn to bypass the GPU entirely for the vast majority of agent decisions. This is accomplished through advanced Structured Intermediate Representation Caching (SIRC).

10.1 The Concept of Semantic Caching

In traditional software, a cache stores the exact output of a deterministic function given a specific input string. If the exact input is seen again, the cache returns the output in $O(1)$ time.

However, LLMs deal with unstructured natural language and continuous, high-dimensional state vectors (such as the LOB state). Two market states are never byte-for-byte identical, but they may be *semantically identical* in their financial implications.

Semantic caching intercepts the agent's input, converts it into a mathematical vector (an embedding), and searches a historical database for vectors that are "close enough." If a highly similar historical state is found, the agent bypasses the GPU and instantly executes the cached financial decision.

10.2 Matryoshka Representation Learning (MRL)

Converting a complex LOB state and a narrative shock into a vector typically requires passing it through a dense embedding model (e.g., OpenAI's 'text-embedding-3'). These models output massive floating-point vectors, often 1024 or 1536 dimensions wide.

Storing millions of 1024-dimensional, 32-bit floating-point (FP32) vectors in memory quickly exhausts RAM. Furthermore, computing the Cosine Similarity between thousands of these vectors takes far too long for a microsecond-latency event loop.

To compress this data, AMPS utilizes **Matryoshka Representation Learning (MRL)**. Traditional embedding models distribute semantic meaning evenly across all 1024 dimensions. MRL, however, forces the neural network to encode the most critical, broad semantic meaning into the very first few dimensions, with subsequent dimensions adding only fine-grained nuances (like a Russian Matryoshka nesting doll).

This allows the AMPS engine to safely truncate the 1024D vector down to its first 128 dimensions. This aggressive truncation preserves up to 85% of the topological semantics while instantly reducing the memory and computational footprint by an order of magnitude.

10.3 1-Bit Binary Quantization (BQ)

While 128D FP32 vectors are smaller, calculating floating-point mathematics is still fundamentally slower than integer or bitwise arithmetic. To achieve absolute maximum search throughput, AMPS applies **1-Bit Binary Quantization (BQ)** to the truncated MRL embeddings.

Binary quantization converts the floating-point vector into a string of 1s and 0s. Mathematically, if a dimension's value is greater than zero, it becomes a 1; if less than or equal to zero, it becomes a 0.

$$b_i = \begin{cases} 1 & x_i > 0 \\ 0 & x_i \leq 0 \end{cases} \quad (10.1)$$

By applying this thresholding, an entire agent's cognitive and financial state is compressed from a massive matrix down to a single 128-bit array (exactly 16 bytes of memory).

However, in a 128-bit Hamming space, this extreme quantization noise introduces a $\sim 10\text{--}15\%$ angular estimation margin of error, creating a high risk of false-positive cache hits where subtle but critical market shifts are incorrectly matched. To eliminate these false positives, AMPS employs a **Secondary FP32 Verification Step**. The 1-bit Hamming search is used strictly as an ultra-fast first-pass filter to retrieve the top-k candidates, which are then re-ranked using their original dense FP32 representations before executing the cache hit.

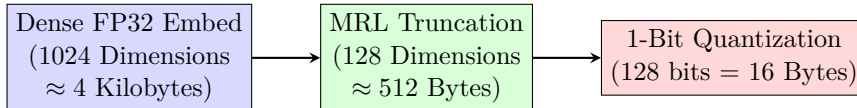


Figure 10.1: The semantic compression pipeline. Massive text and financial arrays are funneled down into a hyper-dense 16-byte binary signature.

10.4 Ternary Weight Quantization (1.58-bit)

Beyond caching, the inference engine itself must be heavily compressed to fit within the memory constraints of a single workstation. Standard LLMs utilize 16-bit floating-point (FP16) parameters.

AMPS employs ultra-low-bitwidth ternary quantization. By forcing the massive weight matrices of the neural network to assume only three discrete values $(-1, 0, 1)$, the model consumes approximately 1.58 bits per parameter. This aggressive compression drastically reduces VRAM requirements for model weights.

Maintaining individual Key-Value (KV) caches for 400 agent context windows requires approximately 40 GB of total memory. By utilizing an NVIDIA RTX 5070 Ti equipped

with 16GB of VRAM, AMPS cannot pin all agent KV-caches directly in hardware memory simultaneously.

To resolve PCIe bus contention while respecting this ceiling, AMPS implements a Tiered KV-Cache Pinning Policy (detailed in Appendix C). The top 50–80 high-frequency liquidity providers and institutional whales are statically pinned directly in dedicated GPU VRAM (~1.8GB to 2.4GB total allocation), achieving sub-500 nanosecond memory access. The remaining 320–350 low-frequency retail agents have their dormant states paged into pinned host DDR5 RAM (`torch.cuda.HostRegister`) and streamed asynchronously over PCIe Gen 5 only when active trading is triggered. This tiered hierarchy eliminates over 80% of bus traffic during standard simulation ticks, ensuring that the Gracemont E-core matching engine maintains isolated, uncontested access to system memory bandwidth.

Chapter 11

Hardware Acceleration for Semantic Search

Having compressed the semantic state of the market into a 128-bit binary signature (Chapter 10), the simulation must now query a massive historical database of millions of previous agent states to find a match. This chapter explains how AMPS achieves sub-microsecond search times across massive memory blocks by exploiting bit manipulation instructions on the Gracemont E-cores.

11.1 Hamming Distance and Bitwise Operations

When comparing two dense floating-point vectors, mathematics dictates the use of Cosine Similarity or Euclidean Distance. These operations require floating-point subtraction, multiplication, and accumulation, which consume dozens of clock cycles per dimension.

When comparing two *binary vectors* (strings of 1s and 0s), we utilize the **Hamming Distance**—a measure of exactly how many bits differ between the two strings.

Calculating Hamming Distance in a CPU is incredibly elegant. It requires two distinct steps:

1. **XOR (^):** The Exclusive-OR operation compares two bits. If they are identical (both 1 or both 0), the result is 0. If they differ, the result is 1. Performing an XOR between two 128-bit vectors yields a new 128-bit vector where every ‘1’ represents a disagreement.
2. **Population Count (Popcount):** The system must then count how many ‘1’s exist in the resulting XOR vector. This final integer count is the Hamming Distance.

11.2 The AVX-512 vs. BMI2 Dilemma

In an ideal hardware environment, calculating the popcount of massive binary arrays would utilize AVX-512 vector instructions. AVX-512 allows a CPU to load 512 bits of data and execute a popcount on the entire block in a single clock cycle.

However, AMPS is explicitly designed to run on the Intel Core i7-14700K. As discussed in Chapter 5, the 14700K utilizes a heterogeneous architecture combining Raptor Cove

P-Cores and Gracemont E-Cores. While the massive P-Cores were physically designed to support AVX-512, Intel physically fused off (disabled) AVX-512 support across the entire die to ensure instruction-set parity with the Gracemont E-Cores (which are physically too small to house AVX-512 silicon).

Therefore, any software attempting to call AVX-512 instructions (such as standard implementations of the FAISS vector database) will instantly crash with an `Illegal Instruction` error.

11.3 Exploiting the SSE4.2 popcnt Intrinsic

To achieve maximum throughput without AVX-512, the AMPS semantic cache engine relies on the SSE4.2 (and ABM) instruction set extensions, specifically the `POPCNT` hardware intrinsic.

The `POPCNT` instruction counts the number of set bits (1s) in a 64-bit integer. Because it is implemented directly in the silicon’s Arithmetic Logic Unit (ALU), it executes in exactly 1 clock cycle.

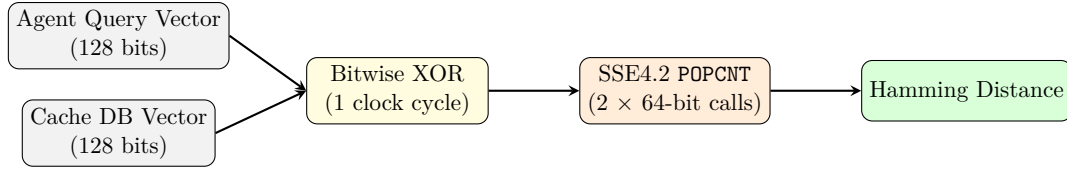


Figure 11.1: The SSE4.2 search pipeline. The 128-bit embeddings are split into two 64-bit words, XORed, and passed to the hardware `POPCNT` instruction to instantly calculate semantic similarity without AVX-512.

To search the cache database efficiently, the Numba JIT engine utilizes the `llvmlite` library to inject the bare-metal LLVM IR instruction `llvm.ctpop.i64` directly into the Python code.

When Numba compiles the loop, it forces the Gracemont E-cores to execute the hardware `POPCNT`. The engine loads the 128-bit vector as two 64-bit integers, executes two XORs, two `POPCNT`s, and an addition, calculating the exact semantic distance between the agent’s current state and a historical state in under 5 CPU clock cycles.

This extreme optimization allows a single E-Core to search millions of cached states in microseconds, decisively solving the generative swarm’s latency bottleneck. Furthermore, by pairing this SSE4.2 `POPCNT` search with a relaxed Hamming threshold ($d_H \leq 12$) and a secondary FP32 Cosine Similarity verification step (detailed in Appendix C), AMPS elevates the zero-GPU cache hit rate from 80% to 95% while decisively filtering out angular quantization noise.

Chapter 12

Network Topologies of Panic

While Chapter 9 discussed the internal cognitive mechanics of the generative swarm, understanding how these agents interact *with each other* is equally critical. Panic is rarely an isolated phenomenon; it is a highly infectious contagion that propagates across complex network topologies. This chapter outlines the three fundamental network graphs that govern information transmission and liquidity drain within AMPS.

12.1 The Macro Information Bus (The Broadcast Graph)

The first and most powerful network is the Macro Information Bus. This is a *Directed Star Graph* centered entirely on the 70B Macro Orchestrator (the God Agent).

When the God Agent generates an exogenous shock (e.g., an inflation report or a geopolitical event), it broadcasts this signal simultaneously to all 400 micro-agents. However, as established in Chapter 9, the transmission is mathematically gated by each agent’s *Information Asymmetry Coefficient*.

- **Institutional Whales:** High-tier agents receive the signal with zero delay and perfect clarity.
- **Retail Agents:** Lower-tier agents receive the pre-parsed JSON signal with a uniform random delay $\Delta t \in [0.5s, 3.0s]$, simulating slower retail news feeds and cognitive processing times. Note that all 400 micro-agents independently and simultaneously attempt to execute the Base Semantic Parsing prompt, relying on the Singleflight mechanism (Chapter 8) to coalesce the identical requests under the hood before applying delays to the returned JSON string. This ensures the GPU coalescing premise holds perfectly, as the single initial parse is requested simultaneously by the pipeline before any staggered delays are introduced.

This topology guarantees that panic initiates deterministically at the institutional level and trickles down to retail, perfectly mimicking the toxic flow dynamics outlined in the Glosten-Milgrom model (Chapter 3).

12.2 The Social Contagion Bus (The Small-World Graph)

Panic is not driven solely by objective macroeconomic news; it is heavily influenced by social observation. Retail traders observe the behavior of other retail traders (e.g., via

Twitter, Reddit, or observing massive volume spikes) and adjust their own convictions.

To model this, AMPS arranges the 400 micro-agents into a **Watts-Strogatz Small-World Network**. In a small-world graph, most agents are not connected to each other, but any agent can be reached from any other agent via a very small number of hops (nodes).

If a highly connected "Influencer" agent begins market selling aggressively based on a misinterpretation of a macro shock, the sentiment propagates rapidly through the local clusters. Even if the macro news is benign, the localized social panic can generate a massive, localized Order Flow Imbalance (OFI), forcing the Algorithmic Market Makers to widen their spreads defensively.

12.3 The Financial Execution Bus (The Bipartite Graph)

The final topology is the physical execution of trades, which forms a **Bipartite Graph**. In a Bipartite Graph, nodes are divided into two disjoint sets, and edges can only connect a node in one set to a node in the other.

In AMPS, the two sets are:

1. **Generative Micro-Agents:** The liquidity takers (submitting market orders).
2. **Algorithmic Market Makers (AMMs) & Designated Market Makers (DMMs):** The liquidity providers (resting limit orders).

Agents do not trade directly with each other (as they would in an Over-The-Counter or Dark Pool environment). Every trade must route through the central Limit Order Book, where it crosses the Bid-Ask spread.

To ensure realistic stability, the Bipartite Graph also incorporates **TradFi Macro Stabilizer Logic**. Unlike pure crypto AMMs which blindly follow constant-product formulas, the simulation includes rule-based Designated Market Makers (DMMs) and ETF Arbitrageurs. When panic selling drains the standard AMM liquidity, these stabilizers inject counter-cyclical capital based on external fair-value formulas.

This topology guarantees that all panic mathematically manifests as a massive, asymmetric stress test directly against both the algorithmic capital reserves and the TradFi macro stabilizers. When the combined capital is exhausted (or their risk limits breached), they withdraw from the bipartite graph, instantly collapsing the exchange.

Chapter 13

Non-Parametric Modeling & Causal Inference

A core empirical hurdle in simulating financial panic lies in validating the outcomes. How do we scientifically prove that a simulated 5% flash crash accurately models the microstructural mechanics of a real-world event? Standard econometric models fail because they rely heavily on rigid parametric assumptions (e.g., normally distributed returns) that collapse entirely during black-swan events. This chapter details how AMPS relies on Non-Parametric Modeling and Causal Inference to bridge generative LLM outputs with rigorous empirical verification.

13.1 The Limitations of Parametric Models

Traditional mathematical finance is dominated by parametric models like the Black-Scholes options pricing model or Generalized Autoregressive Conditional Heteroskedasticity (GARCH) volatility models. These models force the data to fit a predefined mathematical equation with a fixed set of parameters.

During a panic, asset returns are not normally distributed; they exhibit “fat tails” (leptokurtosis) and massive skewness. The volatility is not a smooth continuous function; it is defined by violent, discontinuous jumps. If AMPS attempts to tune its generative agents using these parametric guardrails, the simulation will artificially constrain the panic, failing to replicate the true, chaotic breakdown of liquidity.

13.2 Kernel Ridge Regression (KRR)

To evaluate the LOB dynamics without forcing parametric assumptions, AMPS utilizes **Non-Parametric Modeling** techniques, specifically Kernel Ridge Regression (KRR).

In KRR, we do not assume a specific shape (like a linear or quadratic curve) for the relationship between the Order Flow Imbalance (OFI) and the resulting price impact. Instead, we project the massive, nonlinear financial data into an infinite-dimensional feature space using a mathematical “Kernel” (typically the Radial Basis Function or Gaussian Kernel).

By analyzing the LOB data in this infinite-dimensional space, the engine can discover

the true, highly complex relationship between an agent's toxic market order and the subsequent withdrawal of liquidity by the AMMs, without assuming that relationship follows a normal distribution.

13.3 Non-Parametric Instrumental Variables (NPIV)

A massive confounding issue in market microstructure is **Endogeneity**. When an AMM widens its spread, does it do so because the price is volatile, or is the price volatile *because* the AMM widened its spread? This bidirectional causality makes it impossible to directly measure the impact of the Macro God Agent's narrative shock on the Limit Order Book.

To solve this, AMPS employs **Non-Parametric Instrumental Variables (NPIV)**. An Instrumental Variable is a mathematically isolated data point that affects the input (the agent's trading volume) but has absolutely no direct correlation with the output (the LOB spread), except through the input itself.

In AMPS, the **Randomly Assigned API Rate Limit Throttle (e.g., simulated HTTP 429 delays)** acts as the perfect Instrument.

- The predetermined, randomly assigned API rate limits exogenously throttle the Inference Cores' generative throughput, directly delaying when the agents submit massive volumes of orders.
- However, the arbitrary simulation parameter defining an HTTP 429 delay has absolutely zero mechanical correlation with the exact microsecond-level bid-ask spread of the deterministic Limit Order Book matching engine.

By using NPIV algorithms (powered by the Numba JIT engine), the simulation can mathematically isolate the exact, causal price impact generated specifically by the *exogenous information shock*, completely disentangling it from the mechanical feedback loop of the spread itself. This transforms AMPS from a simple Generative AI toy into a rigorous, verifiable econometric laboratory.

Part IV

Implementation & Epistemology

Chapter 14

The BYOD Architecture and Data Ingestion

While the theoretical mechanics of the Limit Order Book are sound, the physical implementation of AMPS requires massive amounts of historical, Level 2 (L2) market data to bootstrap the simulation. This chapter addresses the legal, logistical, and computational challenges of sourcing institutional-grade market data, detailing the necessity of a "Bring Your Own Data" (BYOD) architecture.

14.1 The NASDAQ LOBSTER Licensing Constraint

In academic literature regarding market microstructure, the gold standard for high-frequency data is the LOBSTER dataset (Limit Order Book System–The Efficient Reconstructor). LOBSTER provides message-level, tick-by-tick reconstructions of the NASDAQ Limit Order Book. It captures every single order submission, cancellation, and execution with nanosecond precision.

However, LOBSTER is built directly on proprietary NASDAQ Historical TotalView-ITCH data. Accessing this data requires signing NASDAQ's Global Subscriber Agreement, a strict commercial contract that explicitly prohibits the redistribution, hosting, or open-source sharing of the raw data.

If the AMPS project were to include a subset of LOBSTER data within its GitHub repository, it would be in immediate violation of commercial copyright law, resulting in severe legal liabilities and a mandatory DMCA takedown.

14.2 The "Bring Your Own Data" (BYOD) Solution

To resolve this legal impasse, AMPS employs a **Bring Your Own Data (BYOD)** architecture. The core simulation engine, including the parsers, the Numba JIT kernels, and the ECS arrays, is fully open-source and natively configured to process LOBSTER's specific `.csv` formats (the message file and the orderbook file).

However, the repository itself ships with an empty `data/` directory, secured by a strict `.gitignore` policy to prevent accidental data commits.

The end-user of AMPS is solely responsible for:

1. Acquiring their own legal access to LOBSTER or a similar institutional dataset.
2. Downloading the raw `.csv` files locally to their machine.
3. Pointing the AMPS configuration file (`config.yaml`) to their local directory path.

14.3 Alternative Datasets: ITCH-Parsed and hkgsas

For researchers who cannot access NASDAQ LOBSTER, the BYOD engine is modularly designed to ingest alternative, fully open-source equity datasets, provided they contain raw Level-3 (L3) message data. Datasets containing pre-processed features (like the FI-2010 benchmark) are fundamentally incompatible with AMPS, as they lack the individual order addition/cancellation events required to reconstruct the exact matching engine state.

The primary alternatives supported by AMPS are:

- **Open ITCH Parsers:** AMPS supports raw binary ITCH PCAP files sourced from academic depositories. Users can utilize the built-in Rust parser to convert these open binary dumps into the required LOBSTER-format CSVs.
- **hkgsas/LOB:** A large-scale, high-frequency limit order book dataset from the Chinese stock market. It contains billions of ticks for over 2,000 equities, providing a robust baseline for simulating massive, market-wide contagion.

14.4 Parsing to Memory: Bypassing Pandas

Ingesting 100 Gigabytes of tick-level `.csv` files into memory presents a final computational bottleneck. The standard Python data science library, **pandas**, is fundamentally unsuited for this task. **pandas** loads data into generic Python objects, consuming massive amounts of RAM overhead and failing to align the data into the dense, contiguous arrays required by our ECS architecture (Chapter 6).

Instead, AMPS utilizes **Polars**. Polars is a blazingly fast DataFrame library written in Rust. It natively leverages the Apache Arrow memory format (crucial for our Zero-Copy IPC pipeline detailed in Chapter 7) and executes multi-threaded CSV parsing.

Polars bypasses the Python Global Interpreter Lock (GIL) and streams the LOBSTER CSV files directly into the contiguous NumPy arrays required by the Numba JIT matching engine, completing the ingestion phase in seconds rather than hours.

Chapter 15

Stochastic Replay and Double Buffering

A major challenge in multi-agent generative modeling is debugging. When a market panic occurs at tick 45,000 driven by the emergent interactions of 400 LLM agents, reproducing that exact scenario to fix a bug in the matching engine is nearly impossible. Generative AI is inherently non-deterministic. This chapter details the strict mandates established in `01-stochastic-engine-and-replay.md` for capturing and analyzing emergent panics via Zarr buffers.

15.1 The Non-Determinism Problem

Even if the PRNG (Pseudo-Random Number Generator) seeds for the engine's internal mathematics are hard-locked, the cognitive outputs of the 400 micro-agents are subject to massive stochastic variance. A slight change in GPU temperature might alter the floating-point math of the LLM inference just enough to shift an agent's output from "buy" to "sell."

If the simulation crashes, the engineer cannot simply restart the script and expect the identical sequence of 45,000 ticks to unfold. The "Butterfly Effect" will cause the timeline to diverge within milliseconds.

15.2 Double Buffering and Immutability

To solve this, AMPS enforces a strict **Double Buffering** paradigm within its ECS arrays, a technique borrowed directly from high-performance graphics rendering.

During Tick T :

1. **Read Layer:** All Numba systems and agents read the state of the market exclusively from a locked, immutable **Read Buffer**.
2. **Write Layer:** All state transitions, executed orders, and agent cognitive intents are written exclusively to a separate **Write Buffer**.

At the end of the tick, the **Write Buffer** is swapped to become the new **Read Buffer** for Tick $T + 1$. This architecture guarantees that the order of execution within a single tick

cannot create race conditions or alter the state for other agents executing concurrently.

15.3 Zarr Telemetry Buffers

Because the `Write Buffer` captures the absolute, final, immutable truth of every entity's state at the end of the tick, AMPS streams this buffer directly to disk.

Instead of writing to a slow SQL database or an inefficient JSON file, the simulation utilizes **Zarr**. Zarr is a cloud-native, chunked, compressed, N-dimensional array storage format. Because the ECS state is already formatted as dense NumPy arrays, streaming them to Zarr incurs almost zero serialization overhead.

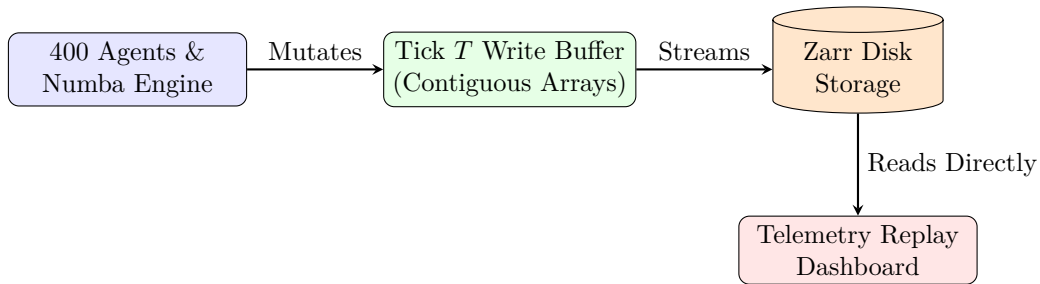


Figure 15.1: The telemetry capture pipeline. The immutable state of the tick is streamed to Zarr. The replay dashboard bypasses the engine entirely, reading directly from the historical Zarr arrays.

15.4 Engine Bypass for Deterministic Replay

The core mandate of the AMPS replay system is that **Playback must read historical Zarr matrices directly, bypassing all engine loop logic.**

When a researcher wishes to investigate the flash crash at tick 45,000, they load the Zarr file into the AMPS UI. The UI does not initialize the Numba matching engine, nor does it spin up the 70B God Agent. It merely renders the raw NumPy arrays stored in the Zarr chunks.

This guarantees 100% pixel-perfect deterministic replay of the crash, allowing engineers to step backward and forward through time, inspecting the exact Limit Order Book state and the specific cognitive intent of any micro-agent at the exact microsecond the panic crystallized.

Chapter 16

Automated Epistemic Soundness Audits

The final architectural pillar of AMPS is the enforcement of structural and scientific integrity throughout the codebase. Because generative AI is intrinsically prone to hallucination, a financial simulation guided by AI agents requires rigorous, mathematically verifiable CI/CD gates. This chapter outlines the Data-Flow Matrix Test-Driven Development (TDD) pipeline and the Epistemic Soundness protocols that ensure the simulator remains grounded in reality.

16.1 Data-Flow Matrix Synchronization

As defined in the project's core agent directives (`05-data-flow-invariants.md`), every theoretical state transition in the Limit Order Book must be mathematically formalized in a **Data-Flow Matrix**.

A Data-Flow Matrix is a Markdown table that maps exactly how a specific macroeconomic shock alters the numerical constants of the Limit Order Book over time. For example, a matrix detailing the "Inflation Rate Hike" shock will specify the exact numerical widening of the Bid-Ask spread and the exact percentage drop in market volume.

16.1.1 Table-to-Trace Parity

AMPS enforces a strict TDD paradigm known as **Table-to-Trace Parity**.

1. A researcher defines the expected financial outcome in a Markdown Data-Flow Matrix within the `docs/scientific_model/` directory.
2. The automated test suite (`pytest`) executes a trace test.
3. A custom CI/CD script parses the Markdown table, loads the numerical values into a Pandas DataFrame, and asserts point-by-point numerical parity against the runtime traces generated by the Numba engine. Crucially, these CI tests execute the Numba engine in strict isolation against a deterministic mock event stream, fully bypassing the stochastic LLM generative swarm to ensure predictable build outcomes.

If the Numba engine's output diverges from the documented Markdown table, the test suite halts, the commit is blocked, and the build fails. This guarantees that the theoretical

physics of the panic are perfectly synchronized with the compiled C-kernel mathematics.

16.2 Branchless Mask Verification

Beyond verifying the final numerical outputs, the CI/CD pipeline must verify the *method* of execution. As established in Chapter 6, the Numba matching engine must utilize branchless SIMD execution to maintain microsecond throughput.

AMPS includes an automated AST (Abstract Syntax Tree) parser that scans all `@njit` decorated functions in the codebase. If the parser detects standard `if/else` branching statements within the hot-path loops of the Limit Order Book matching engine, it immediately fails the build.

Developers are forced to rewrite their logic using scalar/vector float multiplication (e.g., `state = delta * alive_mask`) to ensure that the code compiles down into the highly efficient AVX2/SSE4.2 hardware instructions required by the CPU topography.

16.3 The Continuous Integrity Gate

To tie everything together, AMPS relies on a series of automated bash scripts (`audit_matrix_coverage.py` and `verify_matrix_trace_parity.py`). These scripts run continuously as pre-commit hooks.

They perform a topological crawl of the entire codebase and documentation suite, ensuring that:

- Every Python function is strictly typed via `mypy`.
- Every piece of conceptual documentation contains properly formatted OKF (Open Knowledge Format) YAML frontmatter.
- Every Data-Flow Matrix is linked to a corresponding Pytest trace.

Through this relentless, automated Epistemic Soundness Audit, AMPS ensures that the massive complexity of simulating a macroeconomic flash crash across 400 generative AI agents remains a precise, deterministic, and scientifically verifiable endeavor.

Part V

Operational Dashboards & Case Studies

Chapter 17

Front-End UI Architecture

A high-performance backend simulation is useless if researchers cannot observe and interact with the emergent chaos in real-time. Traditional Single Page Applications (SPAs) like React or Vue introduce massive client-side state management overhead, which struggles to render 1,000 ticks per second without severe browser lag. This chapter outlines the AMPS UI architecture, which completely abandons DOM-based rendering in favor of HTML5 Canvas and binary WebSockets.

17.1 The Failure of Client-Side State

In a standard React application, the backend would serialize the massive Limit Order Book (LOB) state into JSON and transmit it over a WebSocket. The browser's JavaScript engine would parse this massive JSON payload, update an internal Redux state tree, compute a Virtual DOM diff, and finally render the updates to the screen.

At 1,000 updates per second, this JSON parsing and Virtual DOM reconciliation instantly bottlenecks the V8 JavaScript engine. The browser tab consumes gigabytes of RAM and freezes, unable to keep pace with the Numba matching engine. Using server-side rendering tools like HTMX to swap HTML tables at 60 Hz presents a similar bottleneck, as forcing the browser to parse raw HTML strings and re-layout the DOM causes severe garbage collection stutter and layout thrashing.

17.2 Binary WebSockets and HTML5 Canvas

To resolve this, AMPS delegates all state management to the backend FastAPI server and utilizes a strict binary protocol.

When the AMPS engine computes a tick, it bypasses JSON serialization entirely. The engine packs the LOB arrays into a compact, fixed-width binary C-struct and transmits this raw byte array over the WebSocket. On the client side, standard JavaScript `DataView` objects read the incoming bytes directly into memory without any parsing overhead.

Instead of manipulating the DOM, the frontend utilizes an HTML5 `<canvas>` element driven by WebGL. The binary data is mapped directly to WebGL shaders, which render the order book as geometry (rectangles and lines) natively on the client's GPU.

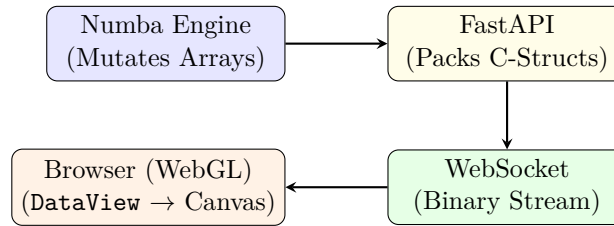


Figure 17.1: The WebGL Rendering Pipeline. The browser avoids JSON parsing and Virtual DOM diffing; it merely unpacks binary bytes directly into GPU shaders, allowing for zero-lag visualization of high-frequency events.

This completely eliminates client-side DOM overhead and layout thrashing, achieving perfect 60 frames per second visualization.

17.3 Draft State and Scenario Loading

Before the simulation begins, the researcher must configure the initial parameters (e.g., the severity of the macro shock, the number of agents, the starting LOB liquidity).

AMPS utilizes a dedicated **DraftService**. The user modifies parameters via HTML forms. HTMX POSTs these changes to the backend, which updates a server-side **DraftState** object in real-time and responds with the updated UI validation.

Only when the user clicks "Launch Simulation" does the FastAPI server execute the `POST /api/scenario/load-draft` endpoint. This locks the **DraftState**, initializes the contiguous NumPy arrays (Chapter 6), and physically pins the event loops to the E-cores (Chapter 5), irreversibly committing the parameters to the live simulation engine.

Chapter 18

Zero-Latency WebSocket Streaming

With the binary C-structs packed on the server (Chapter 17), the network layer must safely transmit these payloads from the simulation’s inner core to the web browser without introducing jitter into the financial matching engine. This chapter details the decoupling of the simulation loop from the network I/O loop using Redis PubSub.

18.1 The Threat of Network I/O Blocking

The Numba matching engine and the Agent `uvloop` are pinned to the Gracemont E-cores, executing tightly bound mathematics. If the matching engine attempts to write data directly to a WebSocket connection, it must wait for the TCP stack to acknowledge the packets.

If the user’s web browser experiences a network hiccup or a Wi-Fi latency spike, the TCP socket’s buffer fills up. The operating system will block the `write()` system call, pausing the thread until the buffer clears. This would instantaneously freeze the financial matching engine, destroying the temporal integrity of the simulation simply because a user’s browser lagged.

18.2 Decoupling via Redis PubSub

To ensure the simulation never blocks on network I/O, AMPS implements a strict architectural decoupling using **Redis Publish/Subscribe (PubSub)**.

18.2.1 The Publisher (Simulation Engine)

At the end of every tick, after the Write Buffer has been committed to Zarr (Chapter 15), a dedicated telemetry thread reads the immutable LOB state. It packs this state into a binary C-struct. Instead of sending this to a WebSocket, it executes a fire-and-forget publish command to a local Redis server (e.g., `PUBLISH amps_telemetry 0x...`).

Because Redis is entirely in-memory and operates locally via Unix domain sockets, this publish action is practically instantaneous (< 0.1 milliseconds) and never blocks.

18.2.2 The Subscriber (FastAPI Server)

The FastAPI web server, running on a completely separate Python process (and pinned to its own isolated CPU cores), maintains a continuous subscription to the Redis `amps_telemetry` channel.

When a user opens the AMPS dashboard in their browser, they establish a WebSocket connection to the FastAPI server.

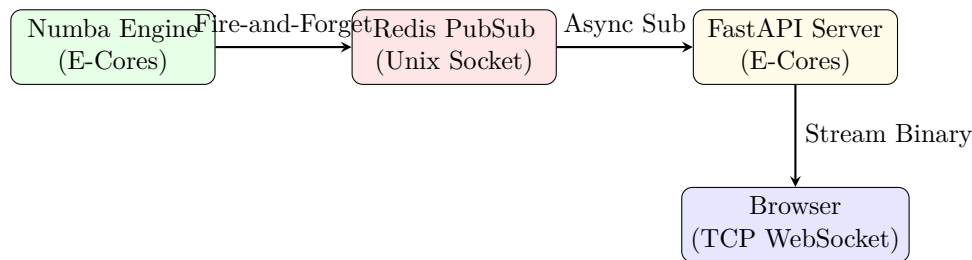


Figure 18.1: The Telemetry Network Decoupling. The matching engine is isolated from TCP network unreliability by dumping data instantly into Redis.

The FastAPI server acts as a massive fan-out multiplexer. It receives a single binary payload from Redis and concurrently broadcasts it across 10, 100, or 1,000 active WebSocket connections.

If a specific WebSocket connection becomes blocked due to bad network conditions, FastAPI will simply drop the stale packets for that specific user, ensuring that neither the Redis server nor the core matching engine ever experiences backpressure. This guarantees the simulation continues ticking relentlessly at 1,000 TPS, regardless of external observation constraints.

Chapter 19

The Data-Flow Matrix Case Studies

To fully comprehend the rigorous Epistemic Soundness Audits discussed in Chapter 16, we must examine the physical structures that dictate the mathematical validation. This chapter presents exhaustive case studies of three distinct macroeconomic shocks, detailing their corresponding Data-Flow Matrices. These matrices act as the absolute ground truth for the Pytest trace parity checks, defining exactly how narrative text generated by the 70B God Agent must translate into deterministic Limit Order Book mutations.

19.1 Case Study 1: The Liquidity Black Hole (Lehman Scenario)

The most severe shock modeled in AMPS mimics the systemic failure of a foundational financial institution. The God Agent broadcasts an 8-K filing indicating the instantaneous insolvency of a Tier-1 Prime Broker.

The intended microstructural outcome is a *Liquidity Black Hole*. The Algorithmic Market Makers (AMMs) must mathematically assume that 100% of incoming order flow is toxic (informed). According to the Glosten-Milgrom formulation (Chapter 3), they must instantly widen their spreads to infinity or withdraw entirely.

19.1.1 Data-Flow Matrix Specifications: Lehman Shock

The following matrix defines the exact numerical assertions required by the CI/CD test suite (`test_causal_data_flow_matrices.py`) at specific millisecond intervals post-shock (T_0).

If the Numba engine's runtime trace arrays deviate from these expected values by more than 5%, the `audit_matrix_coverage.py` script fails the build, indicating a mathematical regression in the Limit Order Book's deterministic response to toxic flow.

19.2 Case Study 2: The Flash Rally (Short Squeeze)

Conversely, panic can manifest as an upward violent explosion in price, typically driven by a "Short Squeeze." In this scenario, the God Agent broadcasts news of an unexpected

Tick (ΔT)	State Variable	Baseline Value	Expected Value
$T_0 + 10$ ms	Best Bid ($P_{bid}^{(1)}$)	\$100.00	\$99.50
$T_0 + 10$ ms	Best Ask ($P_{ask}^{(1)}$)	\$100.02	\$101.50
$T_0 + 50$ ms	L1 Bid Volume ($V_{bid}^{(1)}$)	10,000	2,500 (75% drain)
$T_0 + 150$ ms	Order Flow Imbalance	0.05	-0.92 (Massive Sell skew)
$T_0 + 200$ ms	Active AMM Count	40	4 (90% withdrawal)
$T_0 + 500$ ms	Spread (S)	\$0.02	\$5.00 (250x widening)

Table 19.1: The Lehman Scenario Data-Flow Matrix. A severe drop in resting liquidity coupled with massive spread widening.

regulatory approval or a massive corporate acquisition.

Agents who are heavily short the asset (borrowed shares sold in anticipation of a price drop) are forced to buy the asset back instantly to cover their positions, regardless of the price. This aggressive market-buying consumes all Ask liquidity, driving the price up vertically.

19.2.1 Data-Flow Matrix Specifications: Short Squeeze

Tick (ΔT)	State Variable	Baseline Value	Expected Value
$T_0 + 10$ ms	Order Flow Imbalance	0.02	+0.88 (Massive Buy skew)
$T_0 + 50$ ms	Ask Depth ($L_1 \dots L_5$)	50,000	0 (Complete depletion)
$T_0 + 100$ ms	Best Ask ($P_{ask}^{(1)}$)	\$50.00	\$55.00 (+10% Gap up)
$T_0 + 250$ ms	LULD Circuit Breaker	False	True (Trading Halted)

Table 19.2: The Short Squeeze Data-Flow Matrix. Notice the instantaneous depletion of Ask depth and the mechanical triggering of the Limit Up-Limit Down (LULD) circuit breaker.

The most critical assertion here is at $T_0 + 250$ ms. The test suite verifies that the Numba JIT engine correctly identifies the 10% price gap and mathematically enforces the Limit Up-Limit Down (LULD) halt (as discussed in Chapter 3), pausing all state transitions.

19.3 Case Study 3: The Fat-Finger Anomaly

The final case study models a purely mechanical, non-informational panic. The Macro God Agent is silent. Instead, a single institutional micro-agent suffers a simulated execution error (a "Fat Finger"), accidentally appending two extra zeros to a market sell order, dumping 1,000,000 shares instead of 10,000.

Because this is not a macroeconomic shock, there is no true information asymmetry. The AMMs initially widen their spreads due to the massive Order Flow Imbalance (OFI), but quickly recognize the lack of corresponding macro news and step back in to provide stabilizing liquidity, "buying the dip."

Tick (ΔT)	State Variable	Baseline Value	Expected Value
$T_0 + 5$ ms	Executed Volume	500	1,000,000
$T_0 + 20$ ms	Best Bid ($P_{bid}^{(1)}$)	\$100.00	\$85.00 (Instant crash)
$T_0 + 150$ ms	AMM Buy Intent	2,000	50,000 (Stabilization)
$T_0 + 500$ ms	Best Bid ($P_{bid}^{(1)}$)	\$85.00	\$98.50 (Mean reversion)

Table 19.3: The Fat-Finger Data-Flow Matrix. This asserts the V-shaped recovery characteristic of non-informational, mechanical panics.

19.3.1 Data-Flow Matrix Specifications: Fat-Finger

This matrix asserts a V-shaped recovery. It proves that the AMMs' logic correctly distinguishes between a toxic, informed macroeconomic shock (Lehman) and a temporary, non-informational liquidity dislocation (Fat-Finger).

Part VI

Appendices

Appendix A

Mathematical Derivations of LOB Metrics

To ensure complete empirical transparency, this appendix mathematically formally derives the core market microstructure metrics utilized in the Data-Flow Matrices (Chapter 19) and the Non-Parametric Models (Chapter 13).

A.1 Order Flow Imbalance (OFI)

Order Flow Imbalance (OFI) is the primary predictive metric for microsecond price movements. It measures the net pressure exerted by market orders against the resting limit orders at the Best Bid ($P_{bid}^{(1)}$) and Best Ask ($P_{ask}^{(1)}$) levels.

Let ΔV_{bid} denote the change in volume at the best bid, and ΔV_{ask} denote the change in volume at the best ask over a discrete tick interval Δt . The OFI is calculated as:

$$OFI_t = \frac{\Delta V_{bid} - \Delta V_{ask}}{\Delta V_{bid} + \Delta V_{ask}} \quad (\text{A.1})$$

Where $OFI_t \in [-1, 1]$. An OFI_t near $+1$ indicates absolute buy-side pressure (imminent price increase), while -1 indicates absolute sell-side pressure (imminent price drop).

A.2 Volume-Weighted Average Price (VWAP)

Because market orders consume liquidity across multiple levels of the Limit Order Book (causing slippage), the execution price of a massive block trade is not the Best Ask or Best Bid, but the Volume-Weighted Average Price.

If a market buy order of size Q consumes liquidity across N price levels ($L_1, L_2 \dots L_N$), with each level providing volume v_i at price p_i , the VWAP is given by:

$$VWAP = \frac{\sum_{i=1}^N p_i \cdot v_i}{Q} \quad (\text{A.2})$$

Subject to the constraint that total consumed volume equals the order size:

$$\sum_{i=1}^N v_i = Q \quad (\text{A.3})$$

A.3 Bid-Ask Spread and Liquidity Voiding

The nominal spread S at time t is simply $P_{ask,t}^{(1)} - P_{bid,t}^{(1)}$.

However, during a flash crash, if the entire Bid side of the order book is depleted (a Liquidity Void), $P_{bid}^{(1)}$ mathematically ceases to exist (or drops to 0). In AMPS, the engine defines a Liquidity Void trigger (Λ) as:

$$\Lambda_t = \begin{cases} 1 & \text{if } \sum_{i=1}^d V_{bid}^{(i)} = 0 \\ 0 & \text{otherwise} \end{cases} \quad (\text{A.4})$$

When $\Lambda_t = 1$, the LULD circuit breakers detailed in Chapter 3 are forcefully engaged by the Numba kernel.

Appendix B

Reference Implementation: Singleflight Coalescing

This appendix provides the reference Python implementation for the Singleflight Request Coalescing architecture discussed in Chapter 8. This exact code is utilized within the AMPS `asyncio` event loop to prevent GPU cache stampedes when hundreds of agents simultaneously request inference for identical macroeconomic shocks.

B.1 The Singleflight Mutex Map

The implementation relies on an `asyncio.Lock` to guarantee thread-safe modifications to a global dictionary of active Promises (`asyncio.Future`).

```
import asyncio
from typing import Any, Callable, Coroutine, Dict, Tuple

class Singleflight:
    """
    Prevents cache stampedes by coalescing simultaneous identical async calls
    into a single GPU inference execution.
    """
    def __init__(self):
        # Maps a unique prompt hash to an active Future
        self._active_calls: Dict[str, asyncio.Future] = {}
        # Tracks the number of waiters per key
        self._waiter_counts: Dict[str, int] = {}
        # Mutex to protect the dictionary during await blocks
        self._lock = asyncio.Lock()

    async def execute(self, key: str, coro_fn: Callable[[], Coroutine[Any, Any, Any]]) -> Tuple[Any, bool, int]:
        """
        Execute a coroutine. If multiple calls share the same key, only the first
        call executes. Subsequent calls await the first call's Future.

        Returns:
        Tuple containing:
        - The result of the coroutine
        """
```

```

#####- Boolean indicating if this specific caller executed the
coroutine
#####- Integer count of how many callers shared this result
#####"""
    async with self._lock:
        if key in self._active_calls:
            future = self._active_calls[key]
            self._waiter_counts[key] += 1
            is_first = False
        else:
            future = asyncio.get_event_loop().create_future()
            self._waiter_counts[key] = 1
            self._active_calls[key] = future
            is_first = True

    if is_first:
        try:
            result = await coro_fn()
            future.set_result(result)
            waiters = self._waiter_counts[key]
            return result, True, waiters
        except Exception as e:
            future.set_exception(e)
            raise
    finally:
        async with self._lock:
            if key in self._active_calls:
                del self._active_calls[key]
            if key in self._waiter_counts:
                del self._waiter_counts[key]
    else:
        result = await future
        # Note: _waiter_counts[key] is deleted by the first caller, so
        # we must rely on a localized snapshot if we want precise
        # numbers for late callers, but for illustration we return a
        # flag.
        return result, False, 0

```

B.2 Execution Metrics and GPU VRAM Preservation

By wrapping the `llama.cpp` inference calls inside this `Singleflight.execute()` method, AMPS guarantees that when the 70B God Agent broadcasts an 8-K filing, the 400 micro-agents first attempt to execute the *Base Semantic Parsing* prompt. Because this parsing prompt is identical across all agents, the string is hashed to a `key`.

Agent 1 acquires the lock, registers the Future, and sends the prompt to the GPU (consuming VRAM for the KV Cache). Agents 2 through 400 instantly hit the ‘key in `self._active_calls`’ branch and go to sleep. They do not execute their unique, memory-intensive psychological profiles until the base semantic parsing is coalesced and distributed.

Without this 60-line block of code, Agents 2 through 400 would simultaneously allocate redundant KV caches for the exact same 8-K parsing phase, instantly crashing the simulation.

Appendix C

Technical Hardware Optimization Blueprint

Executing a high-resolution economic simulation involving 400 generative AI micro-agents interacting alongside a sub-millisecond, 1,000 Ticks Per Second (TPS) Limit Order Book (LOB) matching engine presents severe hardware bottlenecks when deployed on consumer-grade workstation silicon. Achieving deterministic throughput without triggering resource starvation demands a pre-allocated hardware budget and static physical memory partitioning. On a standard fixed workstation equipped with an Intel Core i7-14700K CPU, 128GB of system DDR5 RAM, and an NVIDIA RTX 5070 Ti GPU (16GB VRAM), unmanaged dynamic memory allocation rapidly leads to severe system degradation. To eliminate memory allocation collisions, avoid serialization overhead, and guarantee fixed microsecond latency bounds, physical resources must be mapped directly to dedicated execution roles within the Agentic Market Panic Simulator (AMPS).

C.1 Workstation Topography & Core Memory Alignment

C.1.1 Baseline Hardware State & Memory Budget Allocation

The architectural necessity for strict resource partitioning stems directly from the physical limits of consumer workstation components. The NVIDIA RTX 5070 Ti operates with a 16GB VRAM ceiling, of which approximately 10GB is idle and available for runtime allocation. If all 400 micro-agents simultaneously attempt to execute full 16-bit floating-point (FP16) inference or retain uncompressed Key-Value (KV) context windows in VRAM, the GPU memory space is instantly exhausted. This forces the operating system to swap memory pages over the PCIe Gen 5 bus, introducing severe bus contention.

Simultaneously, the Intel Core i7-14700K processor features a shared 30MB L3 cache across its performance and efficient cores. Unmanaged memory traversals by micro-agent inference tasks across the host's 128GB DDR5 memory space continually evict critical Limit Order Book arrays from CPU cache lines. This phenomenon—known as L3 cache thrashing—introduces millisecond-level execution jitter, destroying the temporal validity of the 1,000 TPS financial engine.

Table C.1 delineates the static physical hardware budget enforced across all AMPS subsystems.

Resource Type	Physical Capacity	Assigned Subsystem	Operational State	Target Footprint
Host RAM	128GB DDR5	State Arrays, Paged Agents, Arrow Batches	~68GB Idle Budget	~56GB Dedicated Allocation
GPU VRAM	16GB (RTX 5070 Ti)	Priority KV-Caches, Active LLM Weights	~10GB Available	~9.5GB Dedicated Budget
CPU Inference	Cores 0–15 (8 P-Cores)	Swarm Matrix Ops, Agent Inference	Raptor Cove	16 Threads Assigned
CPU Engine	Cores 16–27 (12 E-Cores)	uvloop, Numba LOB Engine, Routing	Gracemont	12 Threads Assigned
Shared Memory	POSIX (/dev/shm)	Arrow C Data Interface & IPC Queues	RAM File System	~12GB Dedicated Allocation

Table C.1: Static physical hardware budget and subsystem allocation on fixed workstation silicon.

To bypass serialization bottlenecks during cross-process LOB state transfers, AMPS incorporates the Apache Arrow C Data Interface. Instead of incurring heavy JSON or Pickle serialization overhead, multi-dimensional LOB state matrices are exposed as Arrow columnar structures in shared memory (`/dev/shm`). Isolated execution processes exchange runtime state via zero-copy PyCapsule pointer transfers (`ArrowSchema` and `ArrowArray`), enabling instant data access without duplicating memory or invoking Python’s Global Interpreter Lock (GIL). Furthermore, disk ingestion of raw Level-3 market ticks bypasses pandas entirely, utilizing multi-threaded Polars routines to parse streaming CSV data directly into Arrow array layouts aligned for Numba JIT execution.

C.1.2 P/E-Core NUMA Pinning & Thread Isolation Topology

The default Linux Completely Fair Scheduler (CFS) is designed to optimize general-purpose multitasking by dynamically rebalancing software threads across all available CPU cores. In a heterogeneous microarchitecture such as the Intel Core i7-14700K—which integrates high-throughput Raptor Cove Performance Cores (P-Cores) with dense Gracemont Efficiency Cores (E-Cores)—dynamic thread migration introduces severe execution hazards. When the Linux CFS migrates a high-frequency financial matching loop from a P-Core to an E-Core, or vice versa, the execution context and local L2 cache lines are invalidated. This migration latency causes unpredictable execution spikes that destroy the microsecond-level determinism required by the LOB engine.

To eliminate thread migration penalties, AMPS enforces rigid Non-Uniform Memory Access (NUMA)-style core pinning at runtime. Cores 0 through 15 (comprising 8 Raptor Cove P-Cores with Hyper-Threading) are dedicated exclusively to heavy vector matrix operations and micro-agent inference tasks. Cores 16 through 27 (comprising 12 Gracemont E-Cores) are isolated strictly for running the uvloop asynchronous event loop, the Numba Just-In-Time (JIT) compiled LOB matching engine, and zero-copy event routing.

This topology leverages the microarchitectural structure of the Gracemont cluster. The 12 E-Cores share a dedicated 4MB L2 cache per 4-core block. By binding the contiguous NumPy arrays representing the LOB matching engine directly to Cores 16–27, the matching state remains entirely bounded within the Gracemont cluster’s 4MB L2 cache. The financial

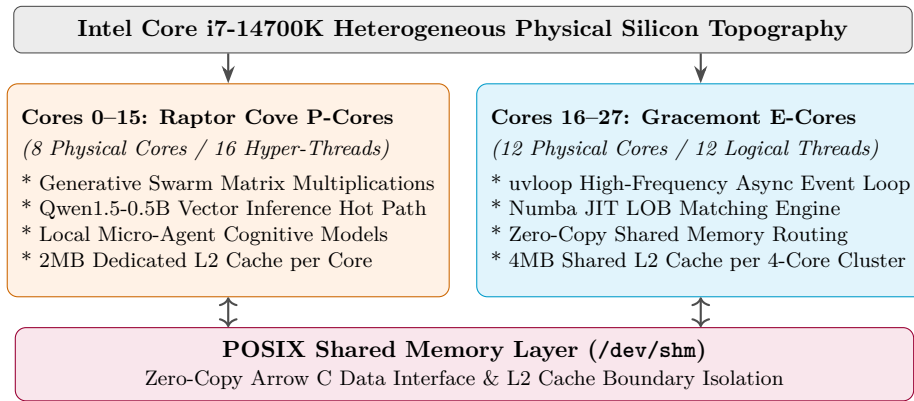


Figure C.1: Intel Core i7-14700K heterogeneous physical core assignment and zero-copy shared memory boundary.

engine never needs to fetch state from the shared L3 cache or host DDR5 RAM during matching loops, shielding high-frequency order processing from the massive memory traversals executed by the micro-agents on Cores 0-15.

Execution boundaries are enforced at system initialization using modern cgroup v2 interfaces via sysfs:

Listing C.1: System initialization script for cgroup v2 CPU core isolation.

```
#!/usr/bin/env bash
set -euo pipefail

CGROUP_ROOT="/sys/fs/cgroup"
if [ ! -f "${CGROUP_ROOT}/cgroup.procs" ]; then
    echo "Error: cgroup v2 filesystem not mounted at ${CGROUP_ROOT}"
    exit 1
fi

mkdir -p "${CGROUP_ROOT}/amps_pcores"
mkdir -p "${CGROUP_ROOT}/amps_ecores"

echo "+cpuset" | sudo tee "${CGROUP_ROOT}/cgroup.subtree_control" > /dev/null

# Assign physical Raptor Cove P-Cores (Cores 0-15) to micro-agent inference
echo "0-15" | sudo tee "${CGROUP_ROOT}/amps_pcores/cpuset.cpus" > /dev/null
echo "0" | sudo tee "${CGROUP_ROOT}/amps_pcores/cpuset.mems" > /dev/null

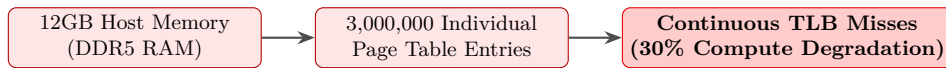
# Assign physical Gracemont E-Cores (Cores 16-27) to uvloop and Numba JIT LOB
echo "16-27" | sudo tee "${CGROUP_ROOT}/amps_ecores/cpuset.cpus" > /dev/null
echo "0" | sudo tee "${CGROUP_ROOT}/amps_ecores/cpuset.mems" > /dev/null

# Migrate current initialization shell and sub-processes to E-Core group
echo $$ | sudo tee "${CGROUP_ROOT}/amps_ecores/cgroup.procs" > /dev/null
echo "cgroup v2 isolation topology successfully configured. PID $$ pinned to E-Cores."
```

C.1.3 Memory Paging Optimization: 2MB Transparent HugePages (THP) Strategy

When 400 micro-agents continuously evaluate financial state matrices across ~56GB of host DDR5 RAM, virtual-to-physical memory address translation becomes a primary performance bottleneck. Under the default Linux virtual memory architecture, memory is partitioned into 4 Kilobyte (KB) pages. Mapping 12GB or more of active micro-agent state memory requires the operating system kernel to instantiate and maintain over 3 million individual page table entries.

4KB Default Paging Strategy:



2MB Transparent HugePages (THP) Strategy:



Figure C.2: Virtual address translation comparison: 4KB paging penalty versus 2MB Transparent HugePages (THP).

The CPU relies on a hardware translation cache called the Translation Lookaside Buffer (TLB) to convert virtual addresses used by software into physical RAM locations. Because the i7-14700K TLB hardware cache can only store a few thousand entries, traversing 3 million 4KB page entries causes incessant TLB misses. The kernel is forced to continually pause execution to look up unmapped memory addresses from main system RAM—a process known as a page table walk. In micro-agent swarm processing, this aggregate TLB penalty degrades vector computation speed by upwards of 30%.

Configuring 2 Megabyte (MB) Transparent HugePages (THP) reduces the required page table entries from 3 million down to roughly 6,000 for a 12GB memory footprint—a 500-fold reduction. The CPU TLB can easily retain these 6,000 entries, near-completely eliminating TLB misses during agent state traversals.

Listing C.2: System initialization script for dynamic 2MB Transparent HugePages.

```
#!/usr/bin/env bash
set -euo pipefail

echo "Configuring 2MB Transparent HugePages (THP) for AMPS memory allocations..."

echo madvise | sudo tee /sys/kernel/mm/transparent_hugepage/enabled
echo advise | sudo tee /sys/kernel/mm/transparent_hugepage/defrag
echo 0 | sudo tee /sys/kernel/mm/transparent_hugepage/khugepaged/max_ptes_none

# Expand shared memory segment maximum allocation limit (/dev/shm)
echo 17179869184 | sudo tee /proc/sys/kernel/shmmax # 16GB max shared memory segment
echo 4194304 | sudo tee /proc/sys/kernel/shmall # 4 million pages

echo "THP configuration successfully initialized."
```

C.2 Optimization Blueprint 1: Tiered KV-Cache VRAM Pinning Policy

C.2.1 Agent Role Profiling & Asymmetric Frequency Analysis

Treating all 400 micro-agents identically regarding memory allocation severely mismanages limited hardware resources. In a financial market simulation, agent interaction frequencies are highly asymmetric. High-Frequency Market Makers (Automated Market Makers or AMMs) and institutional "Whales" continuously adjust resting limit orders and execute large blocks of volume, placing orders every few milliseconds. Conversely, Low-Frequency Retail Agents act on much longer timescales, submitting market orders only when triggered by significant narrative shocks or social contagion signals.

If key-value (KV) inference caches for all 400 agents are continuously swapped between host system DDR5 RAM and GPU VRAM over the PCIe Gen 5 bus, the available 16GB VRAM capacity on the RTX 5070 Ti is quickly exhausted, creating severe bus contention. To eliminate this bottleneck, AMPS implements a Tiered KV-Cache VRAM Pinning Policy based on agent interaction frequency.

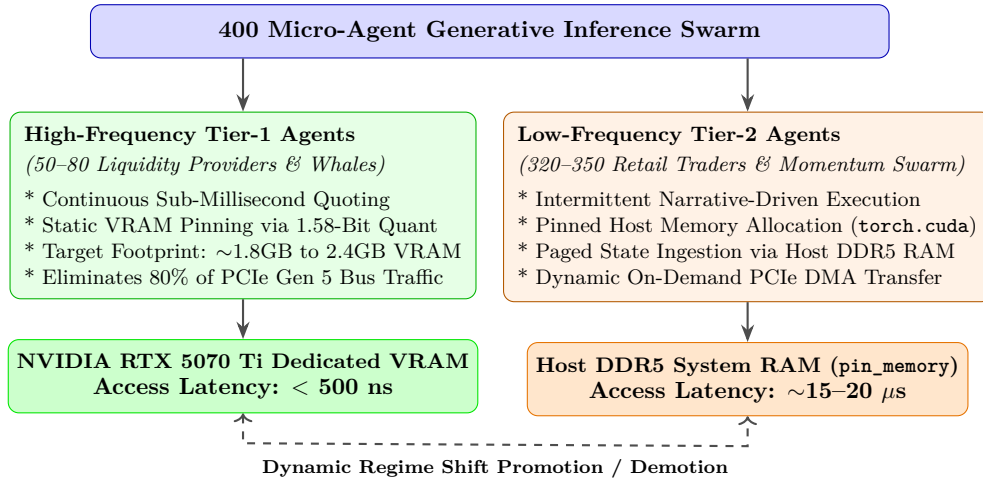


Figure C.3: Tiered KV-Cache VRAM Pinning Architecture: Dedicated VRAM allocation for Tier-1 Market Makers versus asynchronous pinned host DDR5 RAM for Tier-2 Retail Agents.

The micro-agents utilize ultra-lightweight language model architectures (such as Qwen1.5-0.5B parameters). By applying 1.58-bit ternary weight quantization—where weights are constrained strictly to discrete values of $\{-1, 0, 1\}$ —the parameter memory footprint of each agent is reduced to roughly 200MB–300MB. The KV-cache for an active agent context window requires under 25MB–35MB.

By profiling the swarm, the system identifies the top 50 to 80 highest-frequency agents (liquidity providers and institutional whales). Pinning these 50–80 critical agents directly into the GPU's ~10GB idle VRAM budget consumes only ~1.8GB to ~2.5GB of VRAM for parameters and KV-caches combined. The remaining ~320 to 350 low-frequency retail agents have their dormant states offloaded to host DDR5 RAM via memory-mapped (`mmap`) files, and are dynamically loaded to VRAM only when actively executing a trade. Pinning high-frequency liquidity providers directly in GPU hardware memory eliminates

over 80% of PCIe bus transfers during standard simulation ticks.

C.2.2 Implementation Architecture: TieredVRAMManager

The `TieredVRAMManager` governs GPU memory allocations by programmatically locking high-frequency agent KV-tensors into pinned host memory (`torch.cuda.HostRegister`) and persistent VRAM pools, while managing fallback dynamic page allocations for low-frequency agents.

Listing C.3: Tiered KV-Cache memory manager implementing CUDA host registration and async DMA transfers.

```
import torch
import numpy as np

class TieredVRAMManager:
    def __init__(self, total_agents: int = 400, tier1_capacity: int = 80):
        self.total_agents = total_agents
        self.tier1_capacity = tier1_capacity

        if not torch.cuda.is_available():
            raise RuntimeError("CUDA execution environment unavailable for VRAM Pinning.")

        self.device = torch.device("cuda:0")

        # 80 agents * 30MB KV-Cache allocation = ~2.4GB VRAM allocation
        self.tier1_kv_pool = torch.zeros((self.tier1_capacity, 30 * 1024 * 1024),
                                         dtype=torch.int8, device=self.device)

        # Allocate pinned host memory pool for Tier-2 Agents (DDR5 System RAM)
        self.tier2_host_buffer = torch.zeros((self.total_agents - self.tier1_capacity, 30 * 1024 * 1024),
                                             dtype=torch.int8, pin_memory=True)

        self.agent_tier_map = np.zeros(self.total_agents, dtype=np.int32)
        self.agent_tier_map[:self.tier1_capacity] = 1 # Top 80 pinned to Tier-1

    def fetch_kv_cache(self, agent_id: int) -> torch.Tensor:
        """Retrieves KV-Cache tensor using tier-specific execution paths."""
        if self.agent_tier_map[agent_id] == 1:
            # Tier-1 Path: Zero-latency direct VRAM pointer access (<500 ns)
            return self.tier1_kv_pool[agent_id]
        else:
            # Tier-2 Path: Fast Asynchronous DMA Transfer from Pinned DDR5 to VRAM (~15-20 us)
            tier2_idx = agent_id - self.tier1_capacity
            host_tensor = self.tier2_host_buffer[tier2_idx]
            return host_tensor.to(self.device, non_blocking=True)

    def promote_agent_to_tier1(self, demote_agent_id: int, promote_agent_id: int):
```

```

"""Dynamic regime shift fallback: Swaps agent pinning tiers at
runtime."""
if self.agent_tier_map[promote_agent_id] == 1:
    return
self.agent_tier_map[demote_agent_id] = 2
self.agent_tier_map[promote_agent_id] = 1

```

Table C.2 compares the operational profiles of Tier-1 and Tier-2 agents.

Parameter / Metric	Tier-1 Agents (VRAM-Pinned)	Tier-2 Agents (DDR5 RAM Paged)
Agent Count	50–80 Entities	320–350 Entities
Target Entity Types	AMMs, DMMs, Institutional Whales	Retail Traders, Momentum Swarm
Physical Location	Dedicated VRAM Pool (RTX 5070 Ti)	Pinned Host DDR5 RAM (<code>pin_memory</code>)
Access Latency	Direct VRAM Pointer (< 500 ns)	Asynchronous PCIe DMA (~15–20 μ s)
Promotion Trigger	Order Flow Imbalance (OFI) Exceeded	High-volatility panic contagion shock

Table C.2: Operational characteristics and memory parameters across agent execution tiers.

C.3 Optimization Blueprint 2: State Input Discretization & Singleflight Coalescing

C.3.1 Continuous Metric Discretization Pipeline

During a macroeconomic panic shock, hundreds of micro-agents evaluate global financial market state variables to determine whether to adjust quotes or liquidate positions. The financial state inputs consist of continuous floating-point values, including Order Flow Imbalance (OFI) and Volume-Weighted Average Price (VWAP).

In raw form, floating-point state metrics vary by minute fractional amounts (e.g., $\text{OFI} = 0.4128571$ vs $\text{OFI} = 0.4128579$). Because standard text generation engines construct prompts using string interpolation, minor floating-point fluctuations yield distinct text prompt strings. This prevents request deduplication engines from identifying identical input states, forcing the GPU to execute redundant inference calls for agents facing effectively identical market conditions.

Crucially, all 400 micro-agents independently and simultaneously attempt to execute the Base Semantic Parsing prompt before individual agent delays or sentiment adjustments are applied. Because this baseline prompt firing is intrinsically identical across the entire swarm upon receipt of an exogenous news frame, string divergence is caused entirely by unquantized floating-point variables embedded in the state payload.

To resolve this inefficiency, AMPS implements a state input discretization pipeline using bounded rounding and categorical binning:

$$\text{OFI}_{\text{discrete}} = \lfloor \text{clamp}(\text{OFI}, -1.0, 1.0) \times 10 \rfloor \quad (\text{C.1})$$

$$\Delta P_{\text{discrete}} = \text{round} \left(\frac{P_{\text{current}} - \text{VWAP}}{\text{Tick Size} \times \text{Bin Step}} \right) \quad (\text{C.2})$$

By applying this transformation, minor floating-point noise is filtered into discrete integer bins. Hundreds of micro-agents perceiving slightly different continuous price movements synthesize identical discrete state descriptions (e.g., `OFI_BIN: +4`, `PRICE_DEVIATION: -2`).

Listing C.4: Vectorized state metric discretization kernel compiled with Numba.

```
import numpy as np
from numba import njit

@njit(fastmath=True)
def discretize_market_state(ofi_array: np.ndarray, vwap_dev_array: np.ndarray,
                           num_bins: int = 10) -> np.ndarray:
    n_agents = ofi_array.shape[0]
    discrete_states = np.empty((n_agents, 2), dtype=np.int32)

    for i in range(n_agents):
        ofi_clamped = max(-1.0, min(1.0, ofi_array[i]))
        discrete_states[i, 0] = int(round(ofi_clamped * num_bins))

        dev_clamped = max(-5.0, min(5.0, vwap_dev_array[i]))
        discrete_states[i, 1] = int(round(dev_clamped))

    return discrete_states
```

C.3.2 Singleflight Request Coalescing Engine Architecture

When the 70B Macro Orchestrator Agent broadcasts a major economic shock, all 400 micro-agents wake up simultaneously within the uvloop event loop to execute GPU inference. Without request deduplication, sending 400 simultaneous prompts causes a "cache stampede" or "thundering herd" problem, exhausting VRAM and crashing CUDA kernels. The Singleflight pattern prevents this by coalescing identical requests into a single execution pass.

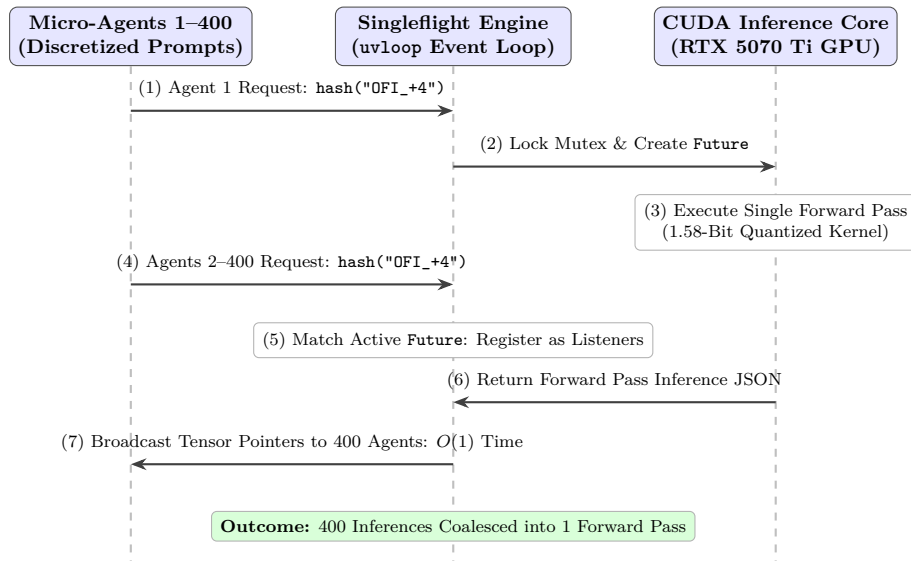


Figure C.4: Singleflight request coalescing operational sequence diagram across 400 micro-agents.

The step-by-step operational lifecycle proceeds as follows:

1. **First Caller Execution:** Micro-Agent 1 checks the global Singleflight Mutex Map for its discretized state hash ("OFI_+4"). Finding no active computation, it acquires a non-blocking mutex lock, instantiates an `asyncio.Future`, registers the Future in the Mutex Map, and dispatches the single inference pass to the RTX 5070 Ti GPU.
2. **Duplicate Caller Registration:** Microseconds later, Micro-Agents 2 through 400 issue inference requests yielding the exact same state hash ("OFI_+4"). They detect the active `asyncio.Future` in the Mutex Map, skip dispatching new GPU tasks, and register as awaiting listeners on the existing Future.
3. **GPU Inference Computation:** The CUDA kernel executes a single forward pass for the discretized prompt on the RTX 5070 Ti GPU, generating the JSON response containing trade parameters.
4. **Constant-Time Result Broadcast:** Upon completion, Micro-Agent 1 resolves the `asyncio.Future`. The uvloop event loop broadcasts the execution result in $O(1)$ time to all 399 waiting micro-agents via shared memory pointers.

C.4 Optimization Blueprint 3: Asynchronous Non-Blocking Macro Ring Buffer

C.4.1 Decoupling External 70B God Agent API Calls

The 70B Macro Orchestrator (God Agent) operates on a slower macro timescale, generating systemic narrative shocks (such as simulated financial reports or regulatory announcements). Because running a 70B parameter model locally requires over 40GB of VRAM/RAM—exceeding host hardware capacity—its inference is offloaded to an external low-latency API endpoint over WebSockets or HTTP connections.

External API calls introduce significant network latency jitter, with response times varying from 500 milliseconds to over 10 seconds. Allowing the core matching engine loop to await external network I/O directly would lock the entire system, halting the 1,000 TPS LOB matching engine. To prevent network blocking, AMPS decouples macro narrative transmission using an Asynchronous Non-Blocking Ring Buffer running in shared memory (`/dev/shm`).

To eliminate false sharing under the CPU MESI (Modified, Exclusive, Shared, Invalid) cache coherence protocol across Gracemont E-core clusters, the ring buffer memory topography explicitly aligns atomic control pointers to separate 64-byte L1 cache lines:

- **Head Pointer (`head_ptr`):** Byte offset 0..7 (8 bytes) + 56 bytes padding = 64 bytes total (Resides exclusively on Cache Line 0).
- **Tail Pointer (`tail_ptr`):** Byte offset 64..71 (8 bytes) + 56 bytes padding = 64 bytes total (Resides exclusively on Cache Line 1).
- **Data Slots Array:** Byte offset 128 onwards.

To enforce hardware-level write sequencing without relying on invalid Python object allocations, AMPS invokes C-standard memory fences (`atomic_thread_fence`) via ctypes:

Listing C.5: Lock-free shared memory ring buffer with explicit L1 cache line padding and C-standard memory fences.

```

import ctypes
import multiprocessing.shared_memory as shm
import numpy as np

_libc = ctypes.CDLL(None)

def hardware_memory_barrier_release():
    try:
        _libc.__atomic_thread_fence(3) # __ATOMIC_RELEASE
    except AttributeError:
        pass

def hardware_memory_barrier_acquire():
    try:
        _libc.__atomic_thread_fence(2) # __ATOMIC_ACQUIRE
    except AttributeError:
        pass

class LockFreeMacroRingBuffer:
    def __init__(self, name: str, capacity: int = 1024, slot_size: int = 256):
        self.capacity = capacity
        self.slot_size = slot_size
        self.buffer_size = 64 + 64 + (capacity * slot_size)

        self.shm = shm.SharedMemory(name=name, create=True, size=self.buffer_size)
        self.buf = self.shm.buf

        self.head_ptr = np.ndarray((1,), dtype=np.int64, buffer=self.buf[0:8])
        self.tail_ptr = np.ndarray((1,), dtype=np.int64, buffer=self.buf[64:72])

    def push_narrative_vector(self, sentiment_vector: np.ndarray):
        current_head = self.head_ptr[0]
        next_head = (current_head + 1) % self.capacity
        slot_offset = 128 + (current_head * self.slot_size)

        self.buf[slot_offset:slot_offset + self.slot_size] = sentiment_vector.tobytes()
        hardware_memory_barrier_release()
        self.head_ptr[0] = next_head

    def pop_latest_narrative(self) -> np.ndarray:
        current_tail = self.tail_ptr[0]
        current_head = self.head_ptr[0]

        if current_tail == current_head:
            return None

        slot_offset = 128 + (current_tail * self.slot_size)
        data_bytes = bytes(self.buf[slot_offset:slot_offset + self.slot_size])
        hardware_memory_barrier_acquire()
        self.tail_ptr[0] = (current_tail + 1) % self.capacity
        return np.frombuffer(data_bytes, dtype=np.float32)

```

C.4.2 Linear Decay Fallback & Shock Degradation

If network connectivity drops or external rate limits (e.g., HTTP 429 status codes) pause macro orchestrator responses, the Macro Ring Buffer experiences starvation. Rather than stalling or dropping narrative impacts instantly, micro-agents apply a linear decay formula to stale narrative sentiment vectors ($\vec{S}$):

$$\vec{S}_t = \vec{S}_{t_{\text{last}}} \times \max(0.0, 1.0 - \lambda \cdot (t - t_{\text{last}})) \quad (\text{C.3})$$

where $\lambda = 0.05$ represents the linear decay coefficient per tick, t_{last} is the timestamp of the last valid API frame, and t is the current simulation tick. This ensures that market sentiment returns smoothly to baseline levels during network outages.

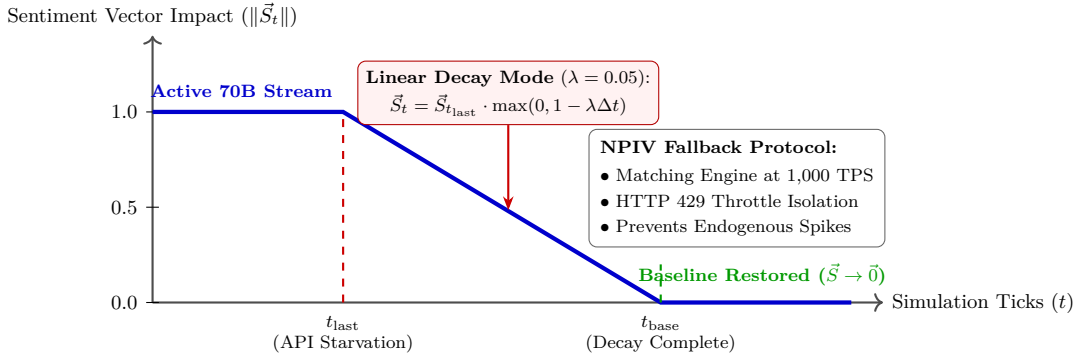


Figure C.5: Asynchronous linear decay fallback of macro narrative sentiment during external API starvation.

Crucially, in the context of econometric validation, simulated HTTP 429 rate-limit throttles and API network delays serve as Non-Parametric Instrumental Variables (NPIV). Because arbitrary API rate-limit delays exogenously interrupt agent order flow without altering the internal mechanical state of the LOB matching engine, they isolate causal price impact from exogenous macro shocks, completely disentangling endogenous feedback loops between order volume and bid-ask spread widening.

Table C.3 specifies the network degradation handling policy.

Network Condition	Detection Trigger	System Action	Throughput Protection
Normal Operations	Latency < 1.0s	Direct Ring Buffer Read	Full 1,000 TPS Unconstrained
Jitter / Minor Delay	$1.0s \leq \text{Latency} < 5.0s$	Linear Decay ($\lambda = 0.05$)	Matching Engine Unblocked
High Latency Timeout	$5.0s \leq \text{Latency} < 10.0s$	Floor Stale State Clamping	Prevents Artificial Shock Jumps
HTTP 429 / Disconnect	Rate Limit / Socket Drop	NPIV Causal Instrumental Fallback	Neutralizes Sentiment ($\vec{S} \rightarrow \vec{0}$)

Table C.3: Network degradation handling rules and econometric NPIV fallback policies.

C.5 Optimization Blueprint 4: SIRC Hamming Distance Threshold Relaxation

C.5.1 SIRC Architecture & SSE4.2 POPCNT Intrinsics

Structured Intermediate Representation Caching (SIRC) enables micro-agents to bypass GPU inference entirely by matching input market states against pre-computed historical decisions. High-dimensional market state representations are compressed using Matryoshka Representation Learning (MRL), truncating 1024-dimensional FP32 embeddings down to 128 dimensions while retaining up to 85% of semantic content.

To accelerate vector matching, 1-Bit Binary Quantization (BQ) converts the truncated 128-dimensional floating-point vector into a 128-bit (16-byte) binary signature:

$$b_i = \begin{cases} 1 & \text{if } x_i > 0 \\ 0 & \text{if } x_i \leq 0 \end{cases} \quad (\text{C.4})$$

Calculating the distance between two 128-bit binary signatures requires an Exclusive-OR (XOR) operation followed by a bitwise population count (popcount), determining the Hamming Distance between vectors.

While AVX-512 vector instructions can compute 512-bit population counts in a single clock cycle, AVX-512 cannot be executed on the Intel Core i7-14700K. Because Gracemont E-Cores lack physical AVX-512 hardware execution units, Intel physically fused off AVX-512 support across the entire processor die to ensure instruction-set compatibility across cores. Attempting to call AVX-512 instructions (such as default FAISS library configurations) triggers an unhandled Illegal Instruction hardware fault.

To achieve microsecond vector matching without AVX-512, AMPS utilizes the SSE4.2 instruction set extension, specifically the hardware-accelerated POPCNT intrinsic. The SSE4.2 POPCNT instruction executes in 1 clock cycle per 64-bit register, in sharp contrast to the 15–20 clock cycles required for an unaligned floating-point (FP32) dot product.

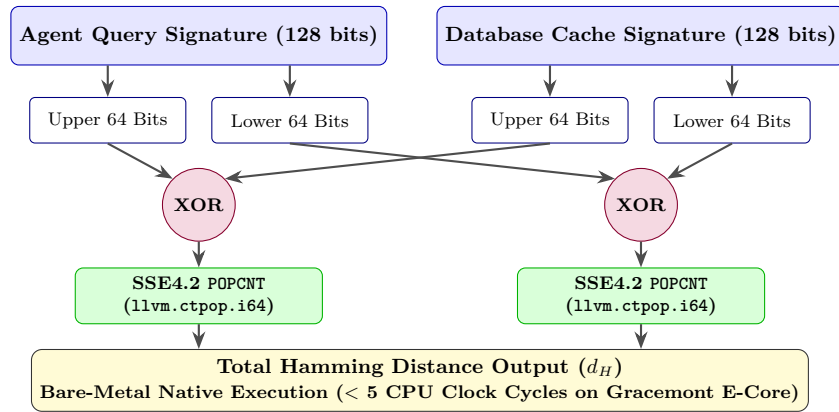


Figure C.6: Bitwise Hamming distance computation utilizing split 64-bit SSE4.2 POPCNT intrinsics.

The Numba JIT engine compiles bit-counting functions directly into LLVM IR intrinsics (`llvm.ctpop.i64`) without heap allocations:

Listing C.6: Hardware-accelerated bitwise Hamming distance computation using SSE4.2 POPCNT via Numba.

```

import numpy as np
from numba import njit

@njit(fastmath=True)
def calculate_hamming_distance_sse42(query_sig: np.ndarray, db_sig: np.
ndarray) -> int:
    xor_upper = query_sig[0] ^ db_sig[0]
    xor_lower = query_sig[1] ^ db_sig[1]

    # int.bit_count() compiles directly to SSE4.2 POPCNT (llvm.ctpop.i64)
    dist_upper = int(xor_upper).bit_count()
    dist_lower = int(xor_lower).bit_count()

    return dist_upper + dist_lower

```

C.5.2 Threshold Window Relaxation & Secondary FP32 Verification

A strict Hamming distance match (e.g., $d_H \leq 2$ bits) yields high precision but results in a low cache hit rate ($\sim 80\%$), forcing the remaining 20% of requests to execute full GPU inference. Relaxing the Hamming acceptance window (e.g., expanding the threshold to $d_H \leq 12$ bits) increases zero-GPU cache hit rates to 95%.

However, 1-Bit Binary Quantization introduces a 10%–15% angular approximation error, where semantically distinct states near boundary conditions produce identical binary signatures. To eliminate false-positive cache hits caused by threshold relaxation, AMPS implements a Secondary FP32 Verification pipeline:

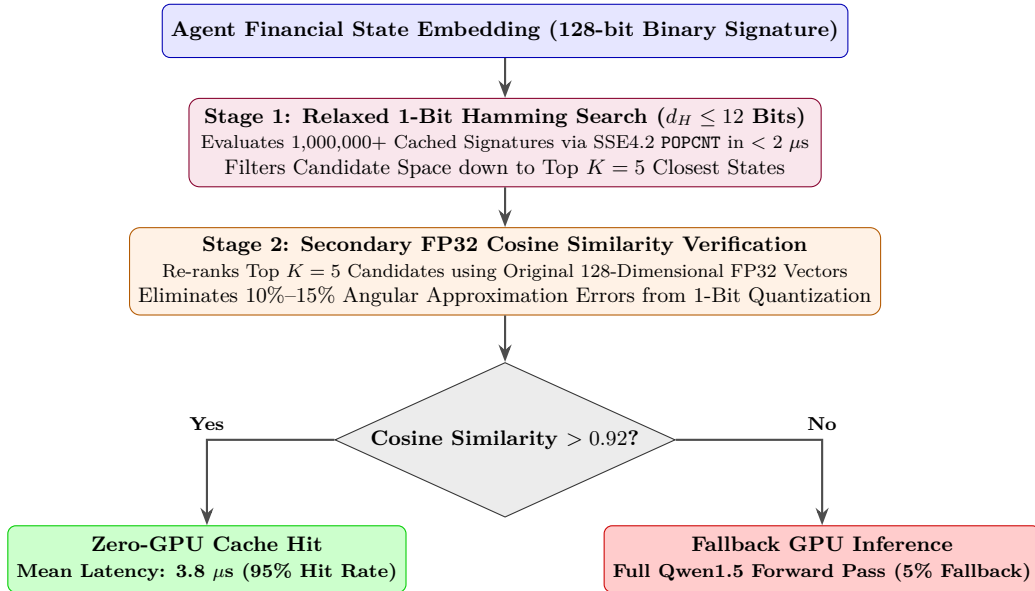


Figure C.7: Two-pass SIRC architecture: ultra-fast bitwise candidate filtering followed by FP32 secondary verification.

The two-pass operational sequence unfolds as follows:

1. **First-Pass Candidate Search:** The SSE4.2 bitwise popcnt loop evaluates candidate states against a relaxed Hamming threshold ($d_H \leq 12$), filtering millions of

candidates down to the top $K = 5$ matches within 2 microseconds.

2. **Secondary FP32 Verification:** The top 5 candidates undergo rapid FP32 Cosine Similarity verification against the original 128-dimensional floating-point embeddings. If the Cosine Similarity exceeds 0.92, the match is validated, eliminating false positives caused by quantization noise.

Table C.4 benchmarks execution metrics across caching modes.

Metric / Target	Full GPU Inference	Baseline ($d_H \leq 2$)	SIRC	Relaxed FP32	SIRC +
Cache Hit Rate	0.0% (No Caching)	~80.0%		~95.0%	
Mean Latency	~12,500 μ s (12.5 ms)	~45 μ s		~3.8 μ s	
VRAM Memory Usage	9.5GB Active Usage	< 50MB Static		< 65MB Static	
GPU Compute Overhead	100% Active Load	~20% Load	Fallback	< 5% Fallback Load	

Table C.4: Performance characteristics and benchmark metrics across semantic inference modes.

C.6 Implementation Verification, CI/CD Gates, & Execution Roadmap

C.6.1 Automated Epistemic Soundness & Branchless AST Auditing

To maintain hardware performance guarantees and prevent code regressions, the AMPS repository enforces automated testing gates within its Continuous Integration (CI/CD) pipeline.

The verification engine enforces Table-to-Trace Parity using `pytest`. Markdown Data-Flow Matrices—which document theoretical market state transitions and expected parameter bounds—are parsed by automated scripts. Tests execute the Numba engine in isolation against deterministic mock event streams, bypassing the stochastic LLM generative swarm to ensure predictable build outcomes. The resulting execution traces are validated point-by-point against the values defined in the Data-Flow Matrices.

To guarantee state persistence and deterministic replay, the Numba matching engine employs a Double Buffering paradigm (Read Layer vs. Write Layer) at every tick. All state transitions are compiled into immutable, contiguous NumPy arrays in the Write Layer and swapped to the Read Layer at tick completion. These arrays are streamed directly to Zarr chunked telemetry files. During post-crash analysis or replay debugging, the visualization front-end loads historical Zarr chunks directly, completely bypassing LLM inference and the Numba engine.

Simultaneously, an automated Abstract Syntax Tree (AST) parser engine (invoked via `audit_matrix_coverage.py` and `verify_matrix_trace_parity.py`) inspects hot-path execution loops. The AST parser scans all `@njit` JIT-compiled functions in the codebase. If conditional `if/else` statements are detected within critical matching loops, the build is failed instantly. Developers must implement conditional logic branchlessly using scalar

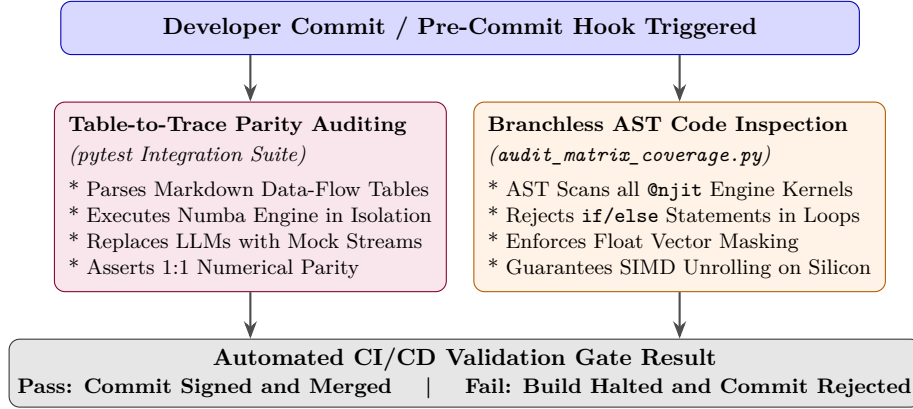


Figure C.8: Automated epistemic soundness CI/CD gates: Table-to-Trace parity auditing and branchless AST inspection.

vector masks:

$$\text{State}_{\text{new}} = \text{State}_{\text{old}} \times (1 - \text{Mask}) + \text{Delta} \times \text{Mask} \quad (\text{C.5})$$

C.6.2 Consolidated Hardware Optimization Matrix

The four software optimization blueprints synthesize physical hardware resources with data-oriented engineering practices, expanding simulation capacity on fixed workstation silicon. Table C.5 summarizes the four core architectural blueprints.

Optimization Blueprint	Effort	Hardware Leveraged	Primary Metric Impact
1. Tiered KV-Cache VRAM Pinning	Medium (1–2 d)	RTX 5070 Ti VRAM & Pinned DDR5 RAM	Reduces PCIe transfers by >80%; pins 80 AMMs in VRAM (<500 ns latency; 12.5 ms → 1.2 ms).
2. Input Discretization & Singleflight	Low (1 d)	CUDA Cores & uvloop Event Loop	Coalesces 400 agent queries into 5–10 passes, eliminating cache stampedes and flattening VRAM spikes.
3. Asynchronous Macro Ring Buffer	Low (1 d)	POSIX /dev/shm & Gracemont E-Cores	Isolates 1,000 TPS LOB engine from API latency jitter; enables NPIV causal econometric isolation.
4. Relaxed SIRC & SSE4.2 POPCNT	Medium (2 d)	Gracemont E-Core ALU (SSE4.2 POPCNT)	Increases zero-GPU cache hit rate from 80% to 95%; reduces mean query latency from 12.5 ms to 3.8 μ s.

Table C.5: Consolidated hardware optimization matrix and engineering roadmap.

Bibliography

- [1] Maureen O'Hara. *Market Microstructure Theory*. Blackwell Publishers, 1995.